
# Mobile Platform (iOS) PlayCtrl Library SDK

Developer Guide

# Contents

# Chapter 1 Overview

## 1.1 Introduction

The Mobile Platform (iOS) PlayCtrl Library SDK is the secondary development kit for the decoding of HIKVISION DVR, DVS and IP devices, etc. It implements the functions of live view, playback, playing control (such as pause, single frame forward/backward, etc), getting stream information (e.g., file indexing information, decoding information, resolution, frame rate, etc), capturing pictures in BMP or JPG format, and so on.

## 1.2 Product Scopes

- NVR: DS-95xx/96xx series and DS-76xx series.
- Hybird DVR: DS-90xx series and DS-76xx series.
- DVR, ATM DVR, Mobile DVR: DS-91xx series, DS-81xx/71xx/72xx series, DS-80xx/70xx series, DS-73xx series.
- DVS, Decoder, Encoder: DS-60xx series, DS-61xx series, DS-63xx series, DS-64xx series, DS-65xx series, and DS-66xx.
- Compression Card: DS-40xx/41xx/42xx/43xx series.
- Network Camera, Network Speed Dome, etc.

## 1.3 System Requirements

Operating System: iOS 6.0 and Above

## 1.4 Conventions

To simplify the descriptions, the Mobile Platform (Android) PlayCtrl Library SDK hereby is referred to “SDK” or “PlayCtrl Library SDK” in the following contents.

# Chapter 2 Update History

### Version Description:

The naming rules of Mobile Platform (iOS) PlayCtrl Library SDK version is described as follows.

***V** Main Version**. Sub Version**. Fix Version**. Reserved Version*

- **Main Version:** Large-scale modification, re-construction or optimization of the SDK
- **Sub Version:** Additional functions/features added to the SDK
- **Fix Version:** Partial changes or bug-fixing of the SDK
- **Reserved Version:** Reserved

***Note:*** If your CPU supports Hyper-Threading Technology, it is recommended to use V3.4 or higher version of the SDK.

## 2.1 Version 7.4.0.X (2020.09)

- Provided new version of IDEMUX library.
- Provided new version of decoding library.
- Added function of anti-aliasing.
- Added function of image post-processing.
- Added function of audio noise reduction.
- Optimized hardware decoding.
- Added function of setting image post-processing parameters, related API: PlayM4_SetImagePostProcessParameter.
- Added function of setting background frame color of the private data with POS information, related API: PlayM4_SetPosBGRectColor.
- Added function of setting whether to reset hardware decoding when exception occurs, related API: PlayM4_SetResetHardDecodeFlag.
- Added function of setting whether to enable multi-slice detection, related API: PlayM4_SetCheckMultiSliceFlag.
- Added function of setting ANR algorithm parameters, related API: PlayM4_SetANRParam.

## 2.2 Version 7.3.9.X (2020.01)

- Added function of enabling feature tracking effect (for EZVIZ Client), related API: PlayM4_EnableSuperEyeEffect.
- Added function of disabling feature tracking effect (for EZVIZ Client), related API: PlayM4_DisableSuperEyeEffect.
- Added function of getting parameters of current display area, related API: PlayM4_GetCurrentRegionRect.
- Optimized codes of fisheye dewarping and display modules.

## 2.3 Version 7.3.8.X (2019.07)

- Added function of setting expected frame rate, related API: PlayM4_SetExpectedFrameRate.
- Added function of setting number of decoding threads, related API: PlayM4_SetDecodeThreadNumber.
- Added function of setting video rendering engine, related API: PlayM4_SetDisplayEngine.
- Added function of setting resolution to capture BMP picture, related API: PlayM4_GetBMPFixPixelEx.
- Added function of setting resolution to capture picture in fisheye mode, related API: PlayM4_FEC_CaptureFixPixel.
- Added function of getting size of picture captured with fixed resolution, related API: PlayM4_FEC_GetCapPicSizeFixPixel.

## 2.4 Version 7.3.7.X (2018.12)

- Added function of setting initialization mode and call mode for Audio Unit, related APIs: PlayM4_SetAudioSessionInit and PlayM4_SetAudioUnitMode.
- Added function of setting absolute timestamp for video playing, related API: PlayM4_SetAbsTimeFlag.
- Added function of setting AGC parameters for audio, related API: PlayM4_SetAGCParam.
- Added function of setting fisheye dewarping parameters for EZVIZ camera, related API: PlayM4_SetLDCFlag.
- Added function of setting chunk size for playing RTMP stream, related API: PlayM4_SetDemuxParam.
- Supports playing stream encrypted by HIK algorithm.
- Supports overlaying private data on picture.
- Added fisheye original view mode.
- Added context sharing mode via iOS EAGL.
- Canceled distinguishing ceil mounting and ground mounting between semisphere view and asteroids view of fisheye dewarping.
- Fixed the bug of security risk in the previous version.
- Fixed the fisheye dewarping bug of EZVIZ cameras.
- Fixed the initial position bug when enabling fisheye dewarping.

## 2.5 Version 7.3.5.X (2018.04)

- Added function of setting decoding tolerance level, related API: PlayM4_SetDecodeERC.
- Added a fisheye dewarping type: FEC_CORRECT_ARC_VERTICAL-180° panorama+vertical curved surface (wall mounting) in the structure FECCORRECTTYPE, related API: PlayM4_FEC_getPort.
- Added function of setting fisheye display callback function, related API: PlayM4_FEC_SetDisplayCallback.
- Added function of playing Intra flash stream.

- Added the external management mechanism for reference frame.

## 2.6 Version 7.3.5.X (2017.11)

### Added:

- Added iOS265 hardware decoding function, processor A9 or above is required, and the version should be iOS11 or above.
- Added software decoding mechanism without copying YUV to optimize the usage of memory.
- Added 3D fisheye dewarping modes: FEC_CORRECT_ARC (horizontal cambered surface), related API: PlayM4_FEC_GetPort.

## 2.7 Version 7.3.4.X (2017.03)

### Added:

- Added drawing callback function (with global time): PlayM4_RegisterDisplayCallBackEx;
- Added decoding callback function (with global time): PlayM4_RegisterDecCallBack;
- Added APIs for enabling or disabling audio sharing: PlayM4_PlaySoundShare, PlayM4_StopSoundShare
- Added API for setting fisheye 3D rotation: PlayM4_FEC_3DRotate;
- Added APIs for fisheye capture: PlayM4_FEC_GetCapPicSize, PlayM4_FEC_Capture;
- Added API for BMP capture of private information: PlayM4_GetBMPEx;
- Added API for reverse play: PlayM4_RecersePlay.
- Set pre-record callback (with global time): setPreRecordCallBackEx;

### Edited:

- Four types of 3D effects are added in PlayM4_FEC_GetPort to the Fisheye module, including FEC_CORRECT_SEM (Hemisphere), FEC_CORRECT_CYC (Cylinder), FEC_CORRECT_PLA (Asteroid) and FEC_CORRECT_CYC_SPL (Cylinder Extended to Plane).

# Chapter 3 Error Code

## 3.1 Common Error Code

| ID | Code | Description |
|---|---|---|
| PLAYM4_NOERROR | 0 | No error |
| PLAYM4_PARA_OVER | 1 | Illegal input parameter |
| PLAYM4_ORDER_ERROR | 2 | Calling reference error |
| PLAYM4_TIMER_ERROR | 3 | Set timer failure |
| PLAYM4_DEC_VIDEO_ERROR | 4 | Video decoding failure |
| PLAYM4_DEC_AUDIO_ERROR | 5 | Audio decoding failure |
| PLAYM4_ALLOC_MEMORY_ERROR | 6 | Memory allocation failure |
| PLAYM4_OPEN_FILE_ERROR | 7 | File operation failure |
| PLAYM4_CREATE_OBJ_ERROR | 8 | Create thread failure |
| PLAYM4_BUF_OVER | 11 | Buffer overflow, input stream failure |
| PLAYM4_CREATE_SOUND_ERROR | 12 | Create sound device failure |
| PLAYM4_SET_VOLUME_ERROR | 13 | Set volume failure |
| PLAYM4_SUPPORT_FILE_ONLY | 14 | This API can only be called in file decoding mode |
| PLAYM4_SUPPORT_STREAM_ONLY | 15 | This API can only be called in stream decoding mode |
| PLAYM4_SYS_NOT_SUPPORT | 16 | System not support, the SDK can only work with CPU above Pentium 3 |
| PLAYM4_FILEHEADER_UNKNOWN | 17 | Missing file header |
| PLAYM4_VERSION_INCORRECT | 18 | Version mismatch between encoder and decoder |
| PALYM4_INIT_DECODER_ERROR | 19 | Initialize decoder failure |
| PLAYM4_CHECK_FILE_ERROR | 20 | File too short or unrecognizable stream |
| PLAYM4_INIT_TIMER_ERROR | 21 | Initialize timer failure |
| PLAYM4_BLT_ERROR | 22 | BLT failure |
| PLAYM4_OPEN_FILE_ERROR_MULTI | 24 | Open video & audio stream failure |
| PLAYM4_OPEN_FILE_ERROR_VIDEO | 25 | Open video stream failure |
| PLAYM4_JPEG_COMPRESS_ERROR | 26 | JPEG compression failure |
| PLAYM4_EXTRACT_NOT_SUPPORT | 27 | File type not supported |
| PLAYM4_EXTRACT_DATA_ERROR | 28 | Data error |
| PLAYM4_SECRET_KEY_ERROR | 29 | Secret key error |
| PLAYM4_DECODE_KEYFRAME_ERROR | 30 | Key frame decoding failure |
| PLAYM4_NEED_MORE_DATA | 31 | Not enough data |
| PLAYM4_INVALID_PORT | 32 | Invalid port number |
| PLAYM4_NOT_FIND | 33 | Search failed. |

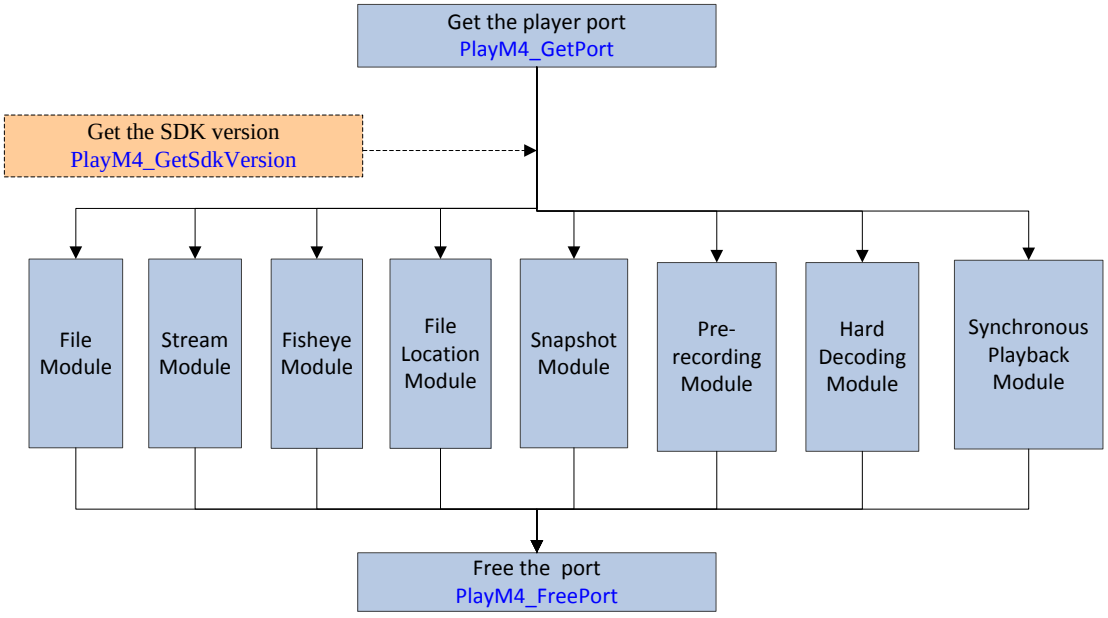
| | | |
|---|---|---|
| PLAYM4_NEED_LARGER_BUFFER | 34 | Insufficient buffer size. |
| PLAYM4_FAIL_UNKNOWN | 99 | Unknown error |

## 3.2 Fisheye Error Code

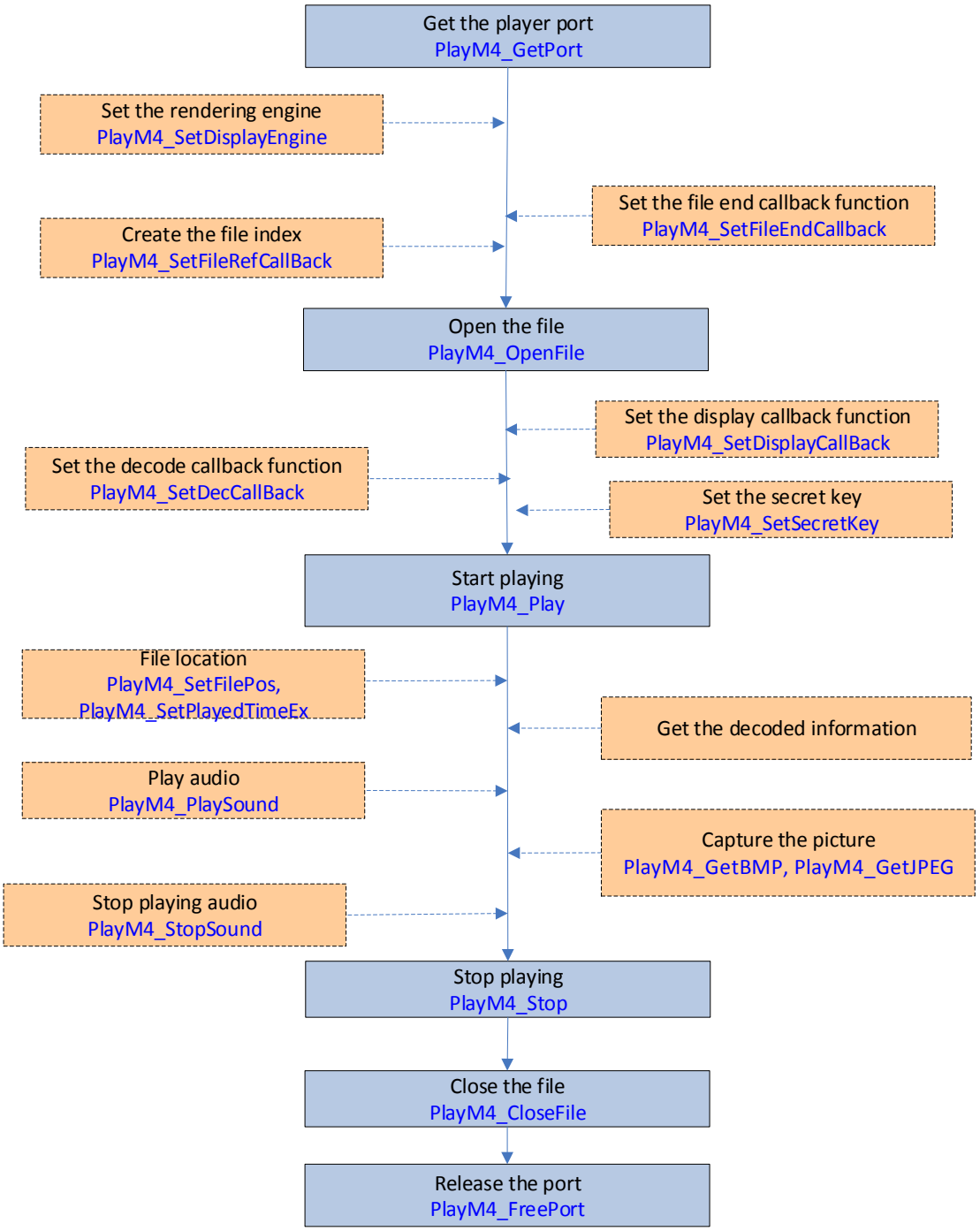
| ID | Code | Description |
|---|---|---|
| PLAYM4_FEC_ERR_ENABLEFAIL | 100 | Load fisheye failure |
| PLAYM4_FEC_ERR_NOTENABLE | 101 | Fisheye is not loaded |
| PLAYM4_FEC_ERR_NOSUBPORT | 102 | Sub-port is not assigned |
| PLAYM4_FEC_ERR_PARAMNOTINIT | 103 | Corresponding port parameters is not initialized |
| PLAYM4_FEC_ERR_SUBPORTOVER | 104 | Sub port is used up |
| PLAYM4_FEC_ERR_EFFECTNOTSUPPORT | 105 | This effect is not supported by this mounting type |
| PLAYM4_FEC_ERR_INVALIDWND | 106 | Illegal window |
| PLAYM4_FEC_ERR_PTZOVERFLOW | 107 | PTZ location crossing |
| PLAYM4_FEC_ERR_RADIUSINVALID | 108 | Illegal circle parameters |
| PLAYM4_FEC_ERR_UPDATENOTSUPPORT | 109 | Specified mounting type and correction effect, this parameter update is not supported |
| PLAYM4_FEC_ERR_NOPLAYPORT | 110 | Playing port is disabled |
| PLAYM4_FEC_ERR_PARAMVALID | 111 | Parameter is NULL |
| PLAYM4_FEC_ERR_INVALIDPORT | 112 | Illegal sub port |
| PLAYM4_FEC_ERR_PTZZOOMOVER | 113 | PTZ correction scope crossing |
| PLAYM4_FEC_ERR_OVERMAXPORT | 114 | Correction channel saturation, maximum 4 channels |
| PLAYM4_FEC_ERR_ENABLED | 115 | This port has enabled fisheye |
| PLAYM4_FEC_ERR_D3DACCENOTENABLE | 116 | D3D acceleration is disabled |
| PLAYM4_FEC_ERR_PLACE_TYPE | 117 | Incorrect mounting type. Each playing port should corresponds to one kind of mounting type. |
| PLAYM4_FEC_ERR_CORRECT_TYPE | 118 | Incorrect dewarping type. |

# Chapter 4 API Call Flow

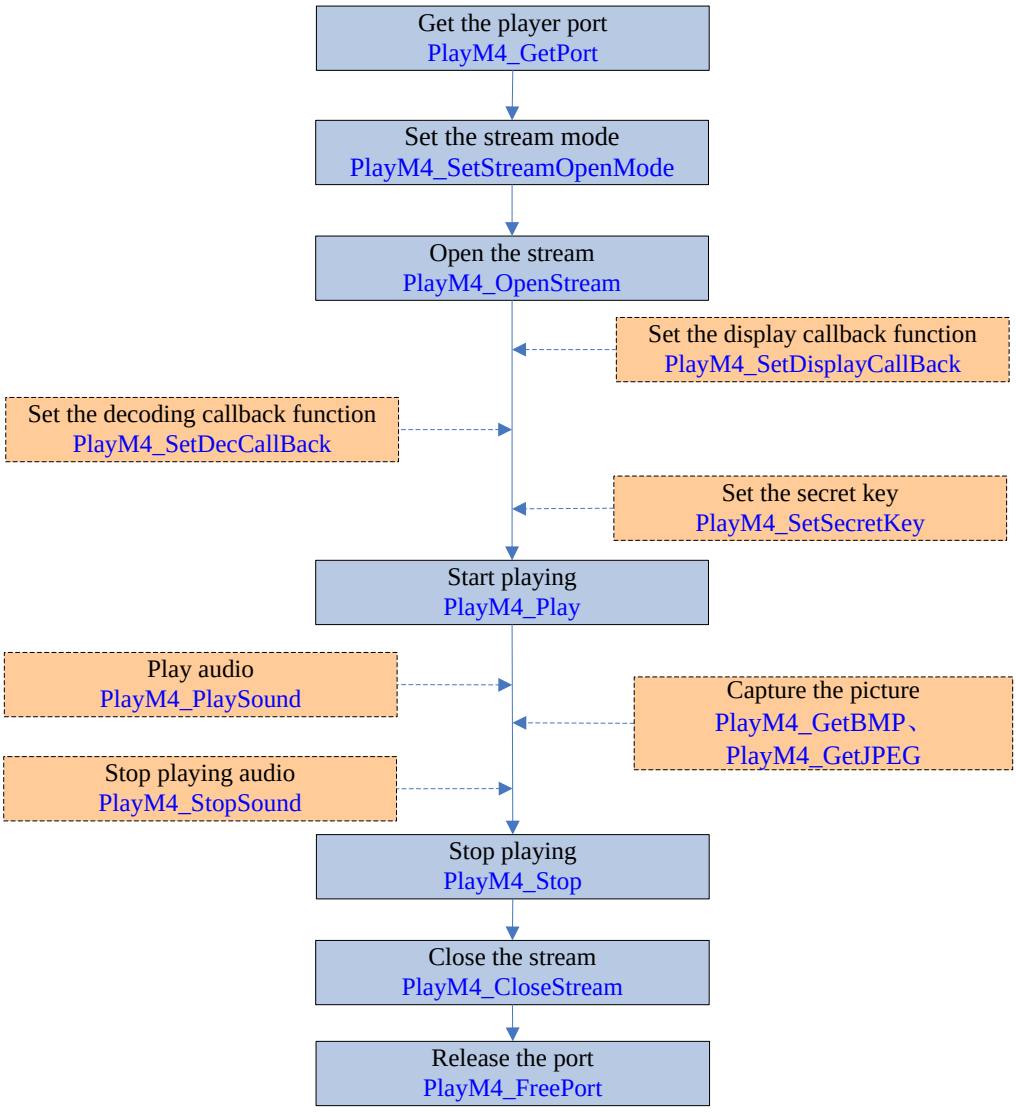
## 4.1 Main Call Progress



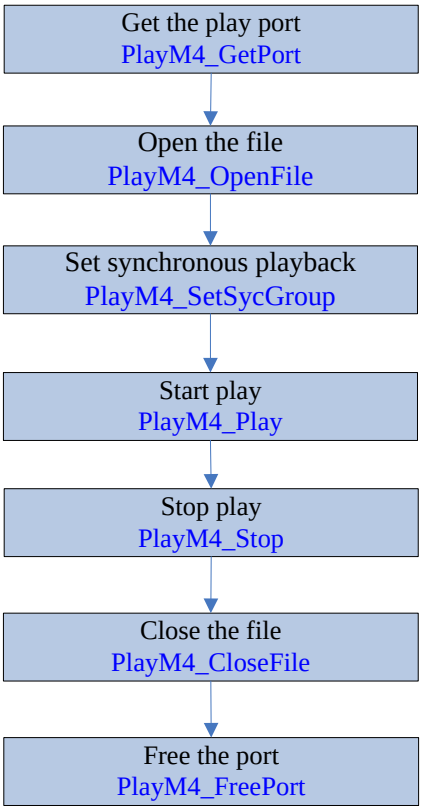
## 4.2 Play in File Mode



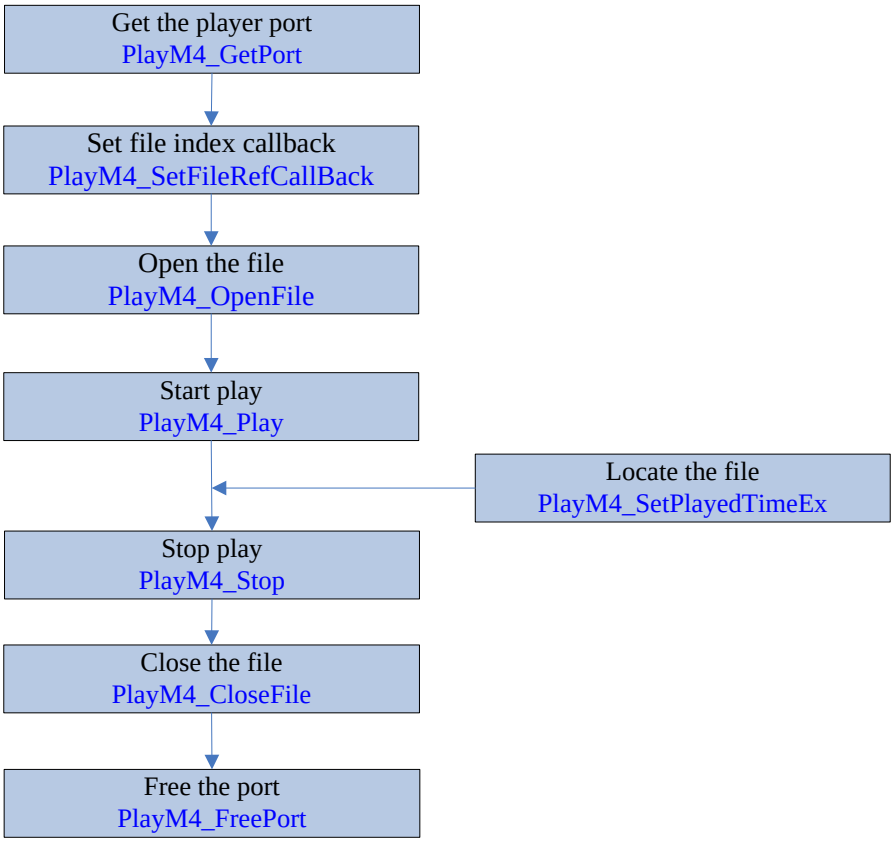
## 4.3 Play in Stream Mode



## 4.4 Synchronous Playback

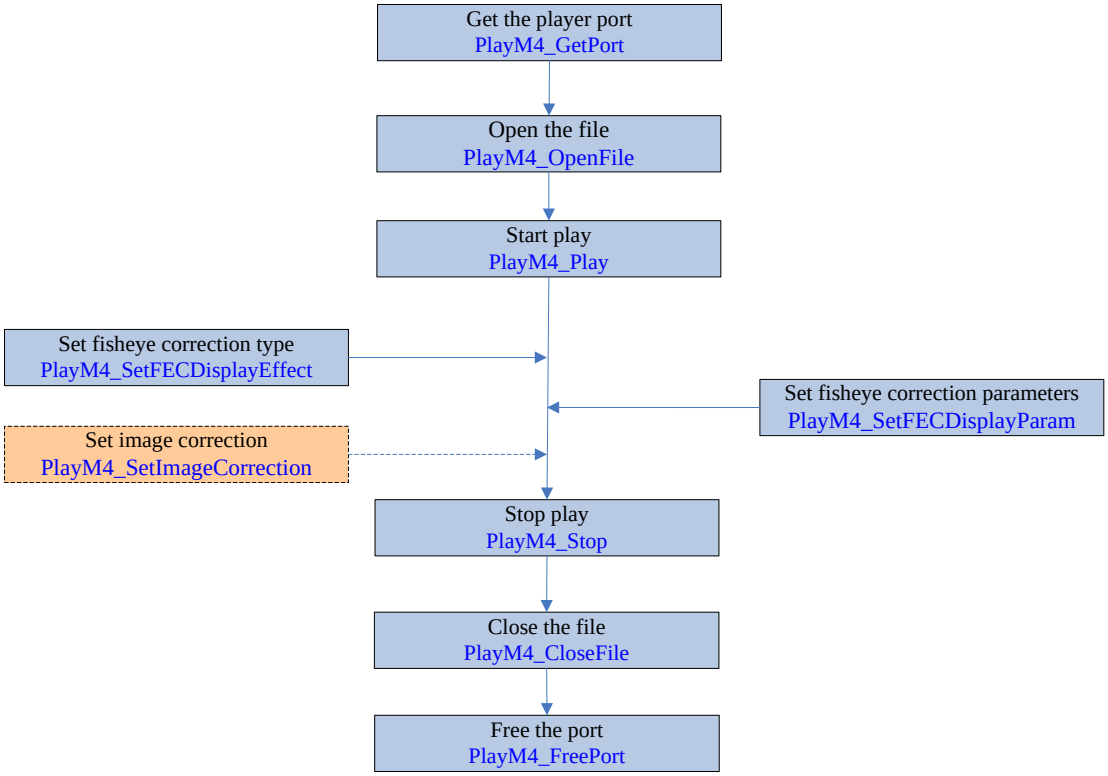


## 4.5 File Locating

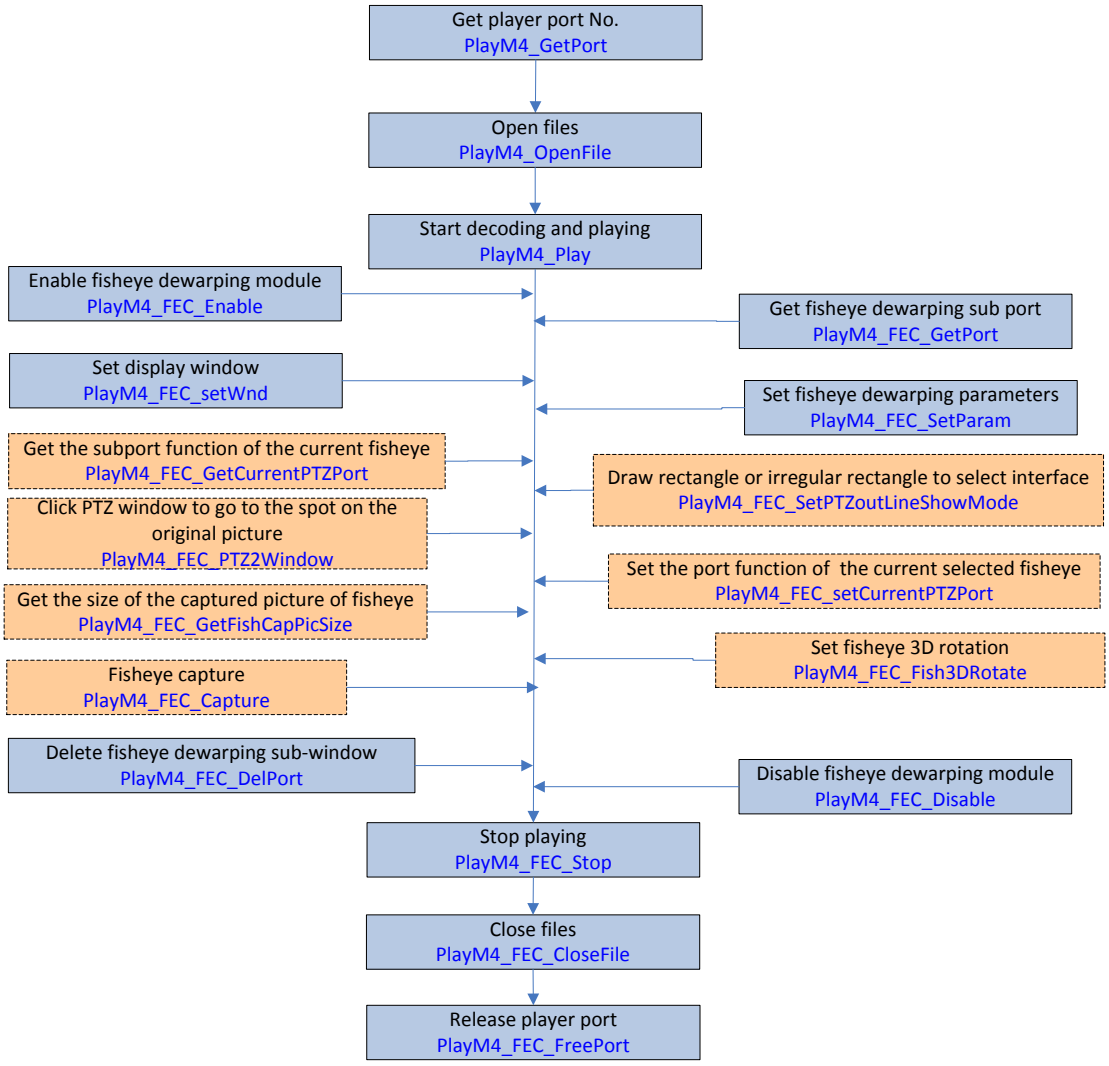


# 4.6 Fisheye

## 4.6.1 Fisheye (Version 7.3.1 and Previous)



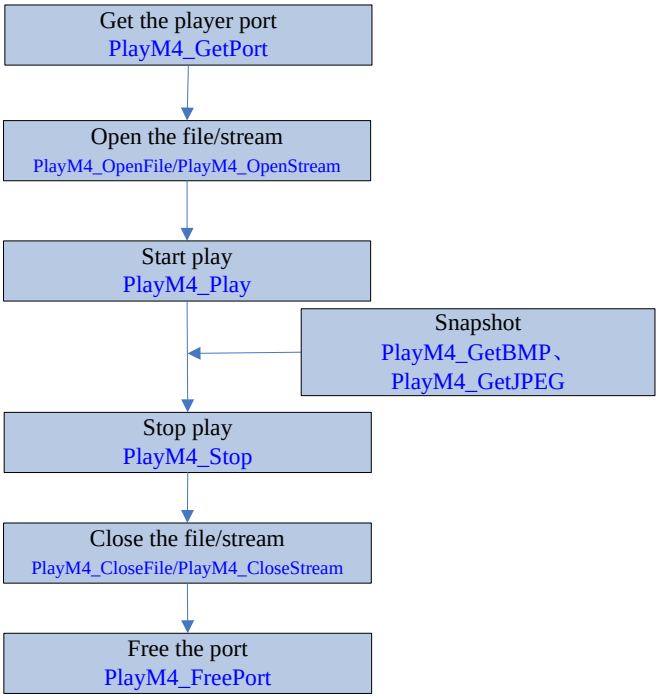
## 4.6.2 Fisheye (Version 7.3.2 and Later)



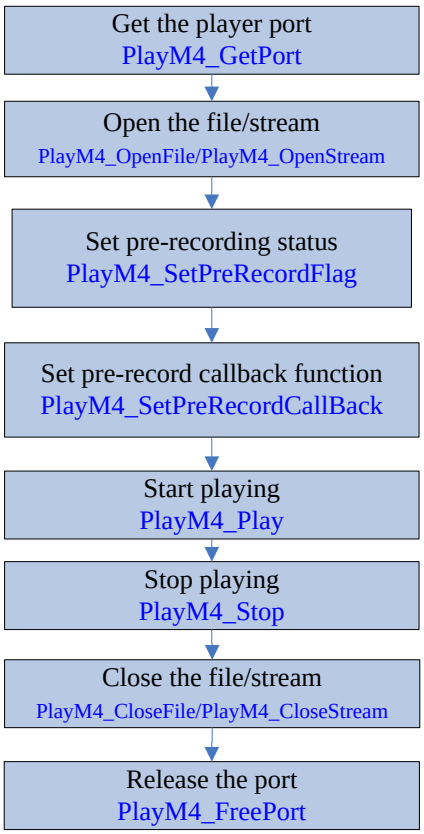

The fisheye module provides the image dewarping of fisheye camera. Before using this module, you should enable live view module, get fisheye dewarping sub port, set fisheye dewarping parameters, set display window and set drawing callback.

- Call API PlayM4_FEC_Enable or PlayM4_FEC_Disable to enable or disable fisheye module
- Call API PlayM4_FEC_GetPort or PlayM4_FEC_DelPort to get or delete fisheye dewarping sub port.
- Call API PlayM4_FEC_SetParam or PlayM4_FEC_GetParam to set or get fisheye dewarping parameters
- The fisheye APIs of version 7.3.1 or pervious and version 7.3.2 or later cannot be called at the same time. Fisheye API of version 7.3.2 or later is suggested.
- Call API PlayM4_FEC_SetWnd to set display window.
- Before calling PlayM4_FEC_GetCapPicSize to realize fisheye capture, you should call PlayM4_FEC_GetCapPicSize to get picture size.
- The above flow chart is based on file mode. While the stream mode also supports fisheye module.
- Fisheye display window cannot be recreated if it is destroyed.

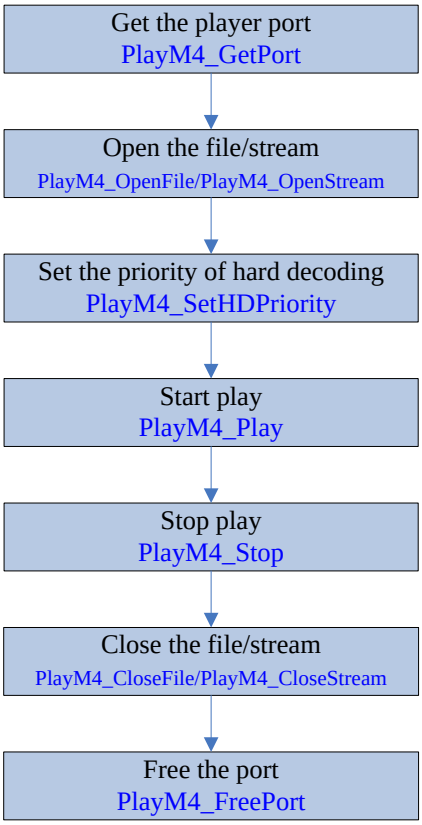
# 4.7 Capture


Get the player port
PlayM4_GetPort
Open the file/stream
PlayM4_OpenFile/PlayM4_OpenStream
Start play
PlayM4_Play
Snapshot
PlayM4_GetBMP、
PlayM4_GetJPEG
Stop play
PlayM4_Stop
Close the file/stream
PlayM4_CloseFile/PlayM4_CloseStream
Free the port
PlayM4_FreePort

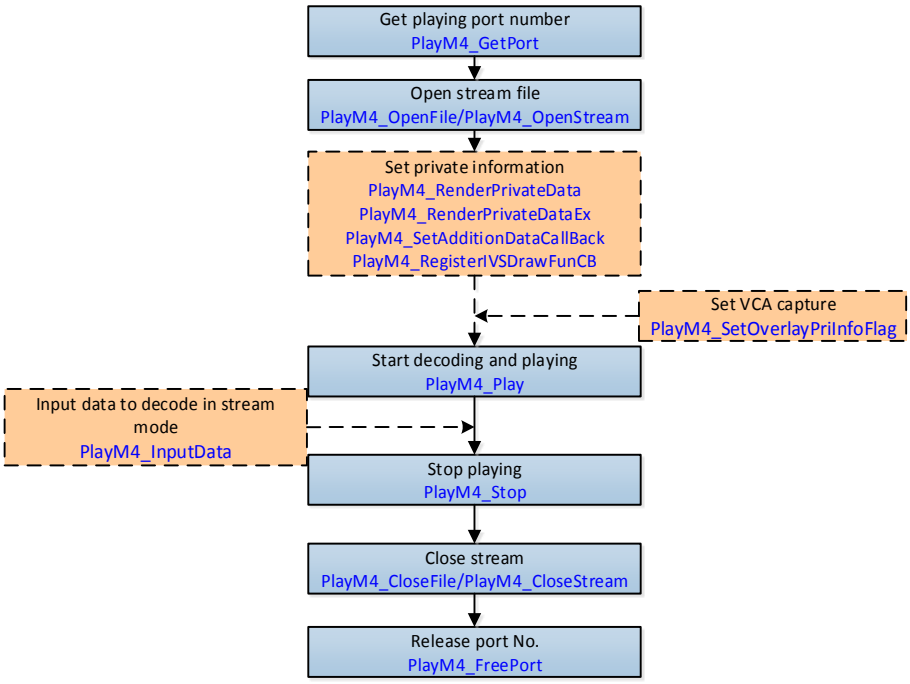
# 4.8 Pre-recording



# 4.9 Hardware Decoding



# 4.10 Display Private Information




- Private Information Module: It is applicable to the stream with VCA information. Otherwise, there is no effect to call this API.
- Set the display of private information (major type: VCA, motion detection, POS information overlay, picture overlay, thermal imaging information): PlayM4_RenderPrivateData.
- Set the display of private information (minor type: when nIntelType is PLAYM4_RENDER_FIRE_DETCET, the value is PLAYM4_FIRE_ALARM; when nIntelType is PLAYM4_RENDER_TEM, the value is PLAYM4_TEM_FLAG).
- Set private information drawing callback: PlayM4_RegisterIVSDrawFunCB.
- Call PlayM4_RenderPrivateData, PlayM4_RenderPrivateDataEx, PlayM4_SetAdditionDataCallBack and PlayM4_RegisterIVSDrawFunCB before or after calling PlayM4_Play; but call PlayM4_SetOverlayPriInfoFlag after the image is decoded, and after calling PlayM4_Play.
- Set capture with private information: PlayM4_SetOverlayPriInfoFlag.

# Chapter 5 API

## 5.1 System Operation and Error Code

### 5.1.1 Get SDK Version Information PlayM4_GetSdkVersion

unsigned int PlayM4_GetSdkVersion();

**Description:**

Get SDK version and build number.

**Return Values:**

Main Version Upgrade: for project structural reorganization or optimizing.

Sub Version Upgrade: for adding new functions.

Editing Version Upgrade: for locally modification or bug fixes.

***Remark:***

For the debug version SDKs, there will only be difference in build number.

### 5.1.2 Get Error Code PlayM4_GetLastError

unsigned int PlayM4_GetLastError (int nPort);

**Description:**

Get the error code.

**Parameters:**

*nPort*

[in] Playing port number

**Return Values:**

The value specified the error code. For the error description, please refer to the table in Chapter 3.

### 5.1.3 Get Play Channel Information PlayM4_GetPort

int PlayM4_GetPort (int* nPort);

**Description:**

Get a valid port number, and the port number should be in the range of [0, 32).

**Parameters:**

**nPort**

[in] [out] int pointer of get the port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.1.4 Release Play Channel PlayM4_FreePort

*int PlayM4_FreePort (int nPort);*

**Description:**

Release the port number which has been occupied

**Parameters:**

**nPort**

[in] Playing port number

It is suggested to set the **nPort** as -1 after a successful port releasing.

**Return Values:**

1- API calling succeeds;

# 5.2 File Operation

## 5.2.1 Open File PlayM4_OpenFile

int PlayM4_OpenFile (int nPort, char*sFileName);

**Description:**

Open the file for playback.

**Parameters:**

**nPort**

[in] Playing port number

**sFileName**

[in] The file name

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remark:***

The file size ranges from 4KB to 4GB.

## 5.2.2 Close File PlayM4_CloseFile

int PlayM4_CloseFile (int nPort);

**Description:**

Close the file that has been opened.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

# 5.3 Stream Operation

## 5.3.1 Set Stream Play Mode PlayM4_SetStreamOpenMode

int PlayM4_SetStreamOpenMode (int nPort, unsigned int nMode);

**Description:**

Set stream input mode.

**Parameters:**

**nPort**

[in] Playing port number

**nMode**

[in] work in stream mode:

**0: STREAME_REALTIME** mode (default)

**1: STREAME_FILE** mode(playback by the time-stamp)

**STREAME_REALTIME** mode gives priority to ensuring of real-time performance and preventing of data blocking problem, and is with strict data checking mechanism while **STREAME_FILE** is the contrary.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remark:***

This API shall be called before playback starts.

## 5.3.2 Open Stream File PlayM4_OpenStream

int PlayM4_OpenStream (int nPort, unsigned char* pFileHeadBuf, unsigned int nSize, unsigned int nBufPoolSize);

**Description:**

Open the stream for playback (similar with open file).

**Parameters:**

**nPort**

[in] Playing port number

**pFileHeadBuf**

[in] The file header which is got from the recording callback APIs of network client SDK or card SDK

**nSize**

[in] The data length of the file header.

**nBufPoolSize**

[in] Specify the size of the source buffer. It ranges from SOURCE_BUF_MIN to SOURCE_BUF_MAX. Users may encounter decoding failure if nBufPoolSize is too small. It is suggested that nBufPoolSize be no less than 200*1024 for SD (standard definition) devices, and no less than 600*1024 for HD (high definition) devices.

```
#define SOURCE_BUF_MAX    1024* 100000
#define SOURCE_BUF_MIN     1024*50
```

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remark:***

```
#define SOURCE_BUF_MAX     1024*100000
#define SOURCE_BUF_MIN     1024*50
```

## 5.3.3 Close Stream File PlayM4_CloseStream

int PlayM4_CloseStream (int nPort);

**Description:**

Close the stream which has been opened.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.3.4 Enter Stream Data PlayM4_InputData

int PlayM4_InputData (int nPort, unsigned char* pBuf, unsigned int nSize);

**Description:**

Input stream data.

**Parameters:**

**nPort**

[in] Playing port number

**pBuf**

[in] buffer address

**nSize**

[in] buffer size

**Return Values:**

If the data input succeeds, the return value is 1.

If the data input fails, the return value is 0. If the error code is 11 is due to the internal buffer full.

***Remark:***

Suggestions for data input failure handling:

- For the data input failure caused by full buffer under **STREAME_REALTIME** mode, users can either discard the data (which will cause frame loss or incomplete decoding video) or retry after sleep of several milliseconds.

- For the data input failure caused by full buffer under **STREAME_FILE** mode, users shall retry until input succeeds.

### 5.3.5 Set Encrypted Stream Callback Function

### PlayM4_SetEncryptTypeCallBack

**API Definition:**

```
int PlayM4_SetEncryptTypeCallBack(
    int                 nPort,
    int                 nType,
    void(CALLBACK* EncryptTypeCBFun)(
        int             nPort,
        ENCRYPT_INFO*   pEncryptInfo,
        void*           nUser,
        int             nReserved2
    ),
    void*               nUser
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No., which is ranging from 0 to 31. |
| *nType* | Encryption callback types. |
| *EncryptTypeCBFun* | Callback function |
| *pEncryptInfo* | Encryption information |
| *nReserved2* | Reserved. |
| *nUser* | User data pointer |

**Return Values:** Return *1* for success, and return *0* for failure.

# 5.4 Playing and Control

## 5.4.1 Start Playing PlayM4_Play

int PlayM4_Play (int nPort, PLAYM4_HWND hWnd);

**Description:**

Start playback. The display image size will be automatically adjusted according to the hWnd window size. For full screen display, please magnify the hWnd window to full screen size.

If the video is already in playback status, calling this API will reset the playback speed to 1X.

**Parameters:**

**nPort**

[in] Playing port number

**hWnd**

[in] The handle of the display window.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.4.2 Stop Playing PlayM4_Stop

int PlayM4_Stop (int nPort);

**Description:**

Stop playback.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.4.3 Pause Playing PlayM4_Pause

int PlayM4_Pause (int nPort, unsigned int nPause);

**Description:**

Pause or resume playback.

**Parameters:**

**nPort**

[in] Playing port number

**nPause**

[in]

1: pause

0: resume the previous playback process

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.4.4 Fast Forward PlayM4_Fast

int PlayM4_Fast (int nPort);

**Description:**

Fast forward. This API can be called up to 4 times continuously to increase the playback speed, which will be doubled after each API call. Call PlayM4_Play() to restore 1X playback from the current position.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remark:***

HD video may not reach the fast forward speed set by the user due to limitation of decoding and display performance.

### 5.4.5 Slow Forward PlayM4_Slow

int PlayM4_Slow (int nPort);

**Description:**

Slow play. This API can be called up to 4 times continuously to decrease the playback speed, which will be lowered by half after each API call. Call PlayM4_Play() to restore 1X playback from the current position.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.4.6 Set Volume PlayM4_AdjustWaveAudio

**Function:** int PlayM4_AdjustWaveAudio(

int nPort,

int nCoefficient

);

| | | | |
|---|---|---|---|
| **Parameters:** | int | nPort | Paying channel No. |
| | int | nCoefficient | Adjustment parameter, 0 means do not adjust, other values are ranging from MIN_WAVE_COEF to MAX_WAVE_COEF. MIN_WAVE_COEF means low volume, MAX_WAVE_COEF means high volume. |
| | *#defineMIN_WAVE_COEF -100* | | *Minimum volume* |
| | *#defineMAX_WAVE_COEF 100* | | *Maximum volume* |

**Return Values:** Return 1 on success, and return 0 on failure.

**Remarks:**

- Call this API to adjust WAVE to change the volume. The difference between this API and PlayM4_SetVolume is that, this API can only adjust the volume of specified channel. While the PlayM4_SetVolume can adjust the sound card volume.
- Adjusting WAVE will destroy the sound quality.
- The settings will take effect after calling this API. Before calling this API, the audio should be turned on.

***Remarks:*** If the system volume is very low, adjusting volume via adjutWaveAudio may have no effect.

### 5.4.7 Play Audio in Exclusive Mode PlayM4_PlaySound

int PlayM4_PlaySound (int nPort);

**Description:**

Open the audio. Only 1-ch audio playback can be enabled, and the audio on other channels will be switched off automatically.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- The audio display is disabled by default.
- PlayM4_PlaySound() and PlayM4_StopSound() shall be used together in the application. If PlayM4_PlaySound() is called, then PlayM4_StopSound() shall be called before application ends.

### 5.4.8 Stop Playing Audio in Exclusive Mode PlayM4_StopSound

int PlayM4_StopSound ();

**Description:**

Stop the audio playback.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.4.9 Play Audio in Sharing Mode PlayM4_PlaySoundShare

**Function:** int PlayM4_PlaySoundShare(
int nPort
)

**Parameters:** *int nPort*
[in] Playing channel No.

**Return Values:** Return 1 on success, and return 0 on failure.

**Remark:**

- Call this API to play audio in sharing mode, while call playSound and stopSound to play audio in monopoly mode.
- playSoundShare, stopSoundShare should be called by pair.
- Calling playSound and stopSound at the same time is not suggested.

**Remark:** By default, the audio data will not be handled if this API is not called.

### 5.4.10 Stop Playing Audio in Sharing Mode PlayM4_StopSoundShare

**Function:** int PlayM4_StopSoundShare(
int nPort
)

**Return Values:** Return 1 on success, and return 0 on failure.

**Remark**:

- playSoundShare and stopSoundShare should be called by pair.
- Calling playSound and stopSound at the same time is not suggested.

**Remark**: PlayM4_PlaySound and PlayM4_StopSound should be called by pair.

### 5.4.11 Play Audio and Video Synchronously

### PlayM4_SyncToAudio

*int PlayM4_SyncToAudio(int nPort, int bSyncToAudio);*

**Description:**

Synchronous playback is based on the time-stamp on the audio and video. If the time-stamp is hoping, this may affect the synchronization.

**Parameters:**

**nPort**

[in] Playing port number

**bSyncToAudio**

[in] Enable synchronous playback or not

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.4.12 Set Auto Flip PlayM4_SetVerticalFlip

*int PlayM4_SetVerticalFlip(unsigned int nPort , int bFlag);*

**Description:**

Set vertical flip of the current picture.

**nPort**

[in] Playing port number

**bFlag**

[in] 1:vertical flip; 0: not vertical flip.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.4.13 Get Playing Progress (Time)

### PlayM4_GetPlayedTimeEx

*unsigned int PlayM4_GetPlayedTimeEx(int nPort)*

**Parameters:**

**nPort**

[in] Playing channel No.

**Return Values:**

Return the current played time on success, unit: ms; return 0 on failure.

**Remarks:**

If the files is be indexed, this API is used as accurate positioning. Otherwise, it is rough

positioning. Accurate positioning is to positioning to the latest video frame, and rough positioning is to positioning to the latest I frame.

### 5.4.14 Set Playing Progress (Time)

#### PlayM4_SetPlayedTimeEx

*int PlayM4_SetPlayedTimeEx (int nPort, unsigned int nTime);*

**Description:**

Set the new position in the file in milliseconds for playback. To get precise locating performance, users need to create file index before executing this operation, otherwise it can only achieve rough locating.

**Parameters:**

**nPort**

[in] Playing port number

**nTime**

[in] Start time for playback

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.4.15 Get Playing Progress (Percentage)

#### PlayM4_GetPlayPos

*float PlayM4_GetPlayPos (int nPort);*

**Description:**

Get the current playback position of the file in percentage.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

The current playback position of the file (percentage)

### 5.4.16 Set Playing Progress (Percentage)

#### PlayM4_SetPlayPos

**API Definition:**

```
int PlayM4_SetPlayPos(
    int     nPort,
    float   fRelativePos
);
```

**Parameters:** *nPort* Play channel No.

*fRelativePos* Play progress (percentage)

**Return Values:** Return *0* for failure, and return *1* for success.

**Remarks:**

- This API should be called after starting live view and enabling fisheye dewarping.
- Before creating index, the locating is rough, if there is no I frame in the located position, the play will be stopped.

## 5.4.17 Get Global Time of Current Played Frame PlayM4_GetSystemTime

**API Definition:**

```
int    PlayM4_GetSystemTime(
   int                    nPort,
   PLAYM4_SYSTEM_TIME     *pstSystemTime
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *pstSystemTime* | Global time, see details in the structure PLAYM4_SYSTEM_TIME. |

**Return Values:** Return *0* for failure, and return *1* for success.

**Remarks:** The obtained system time is in the stream encapsulation layer, which may be different with the OSD time of image.

## 5.4.18 Refresh Display Window PlayM4_RefreshPlay

**API Definition:**

```
int    PlayM4_RefreshPlay(
     int    nPort
);
```

**Parameters:**

*nPort*

[in] Playing channel No.

**Return Values:**

Returns 1 for success, and returns 0 for failure.

**Remark:**

This API is mainly used for refreshing the data frame in both the software decoding and hardware decoding mode.

## 5.4.19 Get Offset of File Online Positioning

## PlayM4_GetMpOffset

**API Definition**

```
int PlayM4_GetMpOffset(
    int     nPort,
    int     nTime,
    int*    nOffset
 );
```

**Parameters**

**nPort**

[in] Playback port

**nTime**

[in] Located play time, unit: ms, range: (0, playing duration)

**nOffset**

[out] Data offset

**Return Values**

Returns 1 for 1 for success, and returns 0 for failure.

**Remarks:**

- This API is mainly used to get the size offset of MP4 data at specific time and locate the file when playing MP4 stream remotely. Playing MP4 file, HIK, PS, and RTP packaging types are not supported.
- When playing MP4 stream, the following stream types are not supported:
  - MP4 stream encoded by MPEG4 is not supported
  - VCA search of MP4 stream is not supported
- See the application flow of this API below:
  1) Open the local file in stream mode and call this API to locate.
  2) Stop playing after locating and wait for the offset data (relocated data).
  3) Restart playing after the offset data is inputted.

  ***Note:*** After calling this API, inputting data must be stopped. And then call resetSourceBuffer to clear the SDK cache. Finally, read the specific offset data again and write it to the SDK.
- User can specify a video time point to start playing, the SDK will return the size of offset data according to the specified time point. And then user resend the data to the SDK according to the data size.

## 5.4.20 Set Squeak Parameters PlayM4_SetHSParam

**API Definition:**

```
Int PlayM4_SetHSParam(
    int     nPort,
    bool    bOpen,
    int     nNotch,
    int     nTime
```

)

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No, which is ranging from 0 to 31 |
| *bOpen* | Enable/disable |
| *nNotch* | Set the filtering level (from 0 to 6), the default level is 4 |
| *nTime* | Set squeak duration, the default is 500. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** This API is customized for EZVIZ client software, so for setting more parameters, you should log in to the EZVIZ client.

## 5.4.21 Set Absolute Timestamp for Video Playing PlayM4_SetAbsTimeFlag

**API Definition:**

```
int    PlayM4_SetAbsTimeFlag(
  int    nPort,
  int    nFlag
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nFlag* | Start or stop playing video by absolute timestamp, 1-start, 0-stop, other values-error. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** This API is only available for EZVIZ products, and the absolute timestamp should be configured via EZVIZ APP.

## 5.4.22 Set AGC Parameters PlayM4_SetAGCParam

**API Definition:**

```
int    PlayM4_SetAGCParam(
  int    nPort,
  int    nEable,
  int    nAGCLevel
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nFlag* | Enable or disable AGC, 1-enable, 0-disable, other values-error. |
| *nAGCLevel* | AGC level, which is between 0 and 30. Level 0: the data is not processed; Level 1 to level 30: the actual loudness is from -32 db to -3 db, that is, when *nAGCLevel* is 30, the loudness is -3 db, when *nAGCLevel* is 29, the loudness is -4 db, and so on. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- This API is customized for EZVIZ products, and the AGC parameters can be configured via EZVIZ client software.
- The audio format supported by this API is EZVIZ AAC (with 16K bps sampling rate).

### 5.4.23 Set Chunk Size for RTMP Stream

**PlayM4_SetDemuxParam**

**API Definition:** int PlayM4_SetDemuxParam(
int *nPort*,
DemuxParam **stParam*
);

**Parameters:**
*nPort* Playing port number
*stParam* Parsing structure, see details in the structure DemuxParam.

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** This API is only available for parsing RTMP stream, and it is required when parsing.

### 5.4.24 Set Initialization Mode for Audio Unit

**PlayM4_SetAudioSessionInit**

**API Definition:** int PlayM4_SetAudioSessionInit(
int *nPort*,
const int *init*
);

**Parameters:**
*nPort* [IN] Playing port number
*init* [IN] Initialization mode, 1-inner initialization (default), 0-external initialization.

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** This API is only used for audio and video AEC (Acoustic Echo Chancellor) process during video intercom.

### 5.4.25 Set Call Mode for Audio Unit

**PlayM4_SetAudioUnitMode**

**API Definition:** int PlayM4_SetAudioUnitMode(
int *nPort*,
const int *init*
);

**Parameters:**
*nPort* Playing port number
*init* Call mode, 1-VOIP (default), 0-normally play.

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** This API is only used for audio and video AEC (Acoustic Echo Chancellor) process during video intercom.

### 5.4.26 Set ANR Algorithm Parameters

**PlayM4_SetANRParam**

**API Definition:**
```
PlayM4_SetANRParam(
    int   nPort,
    int   nEnable,
    int   nANRLevel
    )
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *nEnable* | Whether to enable multi-slice detection. |
| *nANRLevel* | Noise reduction level, which is between 0 and 5, and the default value is 3. |

**Return Values:** Return *true* for success, and return *false* for failure.

# 5.5 Get Playing or Decoding Information

## 5.5.1 Get Video Duration PlayM4_GetFileTime

*unsigned int PlayM4_GetFileTime (int nPort);*

**Description:**

Get total file duration (unit: second).

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

The file's total duration in seconds

## 5.5.2 Get Frame Rate PlayM4_GetCurrentFrameRateEx

*int PlayM4_GetCurrentFrameRateEx(int nPort, float* pfFrameRate);*

**Description:**

Get the current frame rate.

**Parameters:**

**nPort**

[in] Playing port number

**pfFrameRate**

[out] the current frame rate.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.5.3 Get Play Progress (Time) PlayM4_GetPlayedTime

*unsigned int PlayM4_GetPlayedTime (int nPort);*

**Description:**

Get played time count for the current file (unit: second)

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

The played time of the current file, counted from the beginning of the file.

## 5.5.4 Get Original Image Size PlayM4_GetPictureSize

*int PlayM4_GetPictureSize(int nPort,int *pWidth,int *pHeight);*

**Description:**

Get the original image size of the video.

**Parameters:**

**nPort**

[in] Playing port number

**int *pWidth**

[out] original image width, i.e. for CIF/PAL video, the image width is 352

**int *pHeight**

[out] original image height, i.e. for CIF/PAL video, the image height is 288

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- This API gets the size of the latest decoded image. Therefore it shall be called after playback starts to get the precise information.
- It will return the size of 4CIF if the resolution of decoded stream or file is 2CIF.

### 5.5.5 Get Global Time PlayM4_GetSpecialData

*unsigned int PlayM4_GetSpecialData(int nPort);*

**Description:**

Get the global timestamp of current frame.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

If the function succeeds, the return value is a compressed data for global time (unit: second).

```
#define GET_YEAR(_time_)        (((_time_)>>26) + 2000)
#define GET_MONTH(_time_)       (((_time_)>>22) & 15)
#define GET_DAY(_time_)         (((_time_)>>17) & 31)
#define GET_HOUR(_time_)        (((_time_)>>12) & 31)
#define GET_MINUTE(_time_)      (((_time_)>>6)  & 63)
#define GET_SECOND(_time_)      (((_time_)>>0)  & 63)
```

FALSE - API calling fails.

### 5.5.6 Set Expected Frame Rate PlayM4_SetExpectedFrameRate

**API Definition:** int PlayM4_SetExpectedFrameRate(

int *nPort,*

float *fExpectedFrameRate,*

int *nFlag*

);

| | | |
|---|---|---|
| **Parameters:** | *nPort* | [IN] Playing port number, which is between 0 and 31. |
| | *fExpectedFrameRate* | [IN] Expected frame rate |
| | *nFlag* | [IN] Whether to take effect |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- This API must be called after playing the video.
- The expected frame rate is invalid if it is higher than the actural frame rate or lower than 1.

# 5.6 Decoding and Control

## 5.6.1 Set Video Decoding Type

### PlayM4_SetDecodeFrameType

**API Definition**

```
int PlayM4_SetDecodeFrameType(
    int             nPort,
    unsigned int    nFrameType
);
```

**Parameters**

**nPort**

[in] Playback channel No.

**nFrameType**

[in] Frame decoding types, see details below:

| | |
|---|---|
| #define DECODE_NORMAIL | 0-Normal decoding (default) |
| #define DECODE_KEY_FRAME | 1-Decode key frame only |
| #define DECODE_NONE | 2-Not decode video frame |
| #define DECODE_VIDEO_HALF | 3-Decode 1/2 of SVC stream only |
| #define DECODE_VIDEO_QUARTER | 4-Decode 1/4 of SVC stream only |
| #define DECODE_VIDEO_EIGHTH | 5-Decode 1/8 of SVC stream only |
| #define DECODE_ALL_FRAME | 6-Decode all |

**Return Value**

Returns 1 for success, and returns 0 for failure.

**Remark**

For real-time stream, when the decoding type is set to 2, audio stuck may occur.

## 5.6.2 Set Skipping I Frame Decoding

### PlayM4_SetIFrameDecInterval

**Function:** int PlayM4_SetIFrameDecInterval (int nPort, unsigned int dwInterval);

**Parameters:** *nPort*

[in]Playing Channel No.

*iunsigned int dwInterval*

[in]Set the number of I frame interval, meaning the number of I frame shall skip.

**Return:** Return 1 on success, and return 0 on failure.

**Remark:** Configure after setting only decoding I frame.

This API is mainly used for setting I frame decoding. Set setDecodeFrameType(1) as decoding key frame only before calling this API, or the error code 2 will be returned.

### 5.6.3 Set Decoding Callback Function

## PlayM4_SetDecCallBack

*int PlayM4_SetDecCallBack (int nPort, void (CALLBACK* DecCBFun) (int nPort, char * pBuf, int nSize, FRAME_INFO * pFrameInfo, int nReserved1, int nReserved2));*

**Description:**

Register a callback function for user-controllable display.

**Parameters:**

**nPort**

[in] Playing port number

**DecCBFun**

[in] Pointer of the decoder callback function, can't be set as NULL.

***Description of the Callback Function:***

***nPort***

*Playing port number*

***pBuf***

*Pointer of the decoded video/audio buffer*

***nSize***

*Size of pBuf*

***pFrameInfo***

*Pointer of FRAME_INFO structure*

***nReserved1***

*Reserved*

***nReserved2***

*Reserved*

***Description of structure FRAME_INFO:***

```
typedef struct{
  int nWidth;          //Image width in pixels or sound track number
  int nHeight;         // Image height in pixels or audio bit rate
  int nStamp;          //Time stamp in milliseconds
  int nType;           // Received data type as the Macro Definition below
  int nFrameRate;      //Encoding frame rate or audio sampling rate
}FRAME_INFO;
```

**Macro Definition:**

| | |
|---|---|
| **T_AUDIO16** | Sampling rate: 16 Khz, Mono, 16 bits |
| **T_RGB32** | RGB 32 image, 4 bytes per pixel arranged in 'B-G-R-0…' similar as bitmap (starts from the bottom-left of the image) (Reserved) |
| **T_UYVY** | uyvy image, arranged in "U0-Y0-V0-Y1-U2-Y2-V2-Y3…." (starts from the bottom-left of the image) format (Reserved) |
| **T_YV12** | yv12 image arranged in "Y0-Y1-……""V0-V1….""U0-U1-….." format |

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- Currently only T_YV12 format raw video data is supported.
- This API shall be called before **PlayM4_Play** and will be invalid automatically after **PlayM4_Stop.**
- The decoding function provides no speed control, and the decoding starts whenever the call back function returns. To use this function, user shall understand mechanisms of video & audio display.

### 5.6.4 Set Decoding Callback Function with User Transfer Parameter PlayM4_SetDecCallBackMend

*int PlayM4_SetDecCallBackMend (int nPort, void (CALLBACK*DecCBFun) (int nPort, char * pBuf, int nSize, FRAME_INFO * pFrameInfo, int nUser, int nReserved2), int nUser);*

**Description:**

Register a callback function to replace the displayed part. It is controlled by user. It is called back before **PlayM4_Play** and will be invalid automatically when **PlayM4_Stop**. Also it needs to be reset before recalling back **PlayM4_Play**. Please be aware that the decoded part does not control speed, and as user returns from call back function, the decoder will decode the next part of data. To use this function, user must understand video display and audio play. Please refer to direct development package about the relative knowledge.

**Parameters:**

**nPort**

[in] Playing port number

**DecCBFun**

[in] Pointer of the decoder callback function, can't be set as NULL.

***Description of the Callback Function:***

***nPort:***

*Playing port number*

***pBuf:***

*Pointer of the decoded video/audio buffer*

***nSize:***

*Size of pBuf*

***pFrameInfo:***

*Pointer of FRAME_INFO structure*

***nUser:***

*User data*

***nReserved2:***

*Reserved*

***Description of structure FRAME_INFO:***

```
typedef struct{
  int nWidth;              //Image width in pixels or sound track number
  int nHeight;          // Image height in pixels or audio bit rate
```

```
    int nStamp;        //Time stamp in milliseconds
    int nType;         //Received data type as the Macro Definition below
    int nFrameRate;    //Encoding frame rate or audio sampling rate
}FRAME_INFO;
```

**Macro Definition:**

| | |
|---|---|
| **T_AUDIO16** | Sampling rate: 16 Khz, Mono, 16 bits |
| **T_RGB32** | RGB 32 image, 4 bytes per pixel arranged in 'B-G-R-0…' similar as bitmap (starts from the bottom-left of the image) (Reserved) |
| **T_UYVY** | uyvy image, arranged in "U0-Y0-V0-Y1-U2-Y2-V2-Y3…." (starts from the bottom-left of the image) format (Reserved) |
| **T_YV12** | yv12 image arranged in "Y0-Y1-……""V0-V1…""U0-U1-….." format |

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- Currently only T_YV12 format raw video data is supported.
- This API shall be called before **PlayM4_Play** and will be invalid automatically after **PlayM4_Stop.**
- The decoding function provides no speed control, and the decoding starts whenever the call back function returns. To use this function, user shall understand mechanisms of video & audio display. Please refer to DirectX development package about the relative knowledge.

## 5.6.5 Set Decoding Callback Function with User Transfer Parameter PlayM4_RegisterDecCallBack

**Function:**

```
int PlayM4_RegisterDecCallBack(
  int nPort,
  void (CALLBACK* DecCBFun)(
    int      nPort,
    char     *pBuf,
    int nSize,
    FRAME_INFO *pFrameInfo,
    PLAYM4_SYSTEM_TIME *pstSystemTime,
    void* nUser
    ),
    void* nUser
)
```

**Parameters:** *nPort*

[in] Playing channel No.

*DecCBFun*

[in] Decoding callback function. It cannot be Null.

*nUser*

[in] User data

***Decoding Callback Function Parameters:***

| | | |
|---|---|---|
| int | nPort | Playing channel No. |
| char* | pBuf | Decoded video and audio data. |
| int | nSize | Length of decoded video and audio data. |
| FRAME_INFO *pFrameInfo | | Video and audio information |
| PLAYM4_SYSTEM_TIME *pstSystemTime | | System time |
| void* | nUser | User data |
| void* | nReserved2 | Reserved parameter |

**Return:** Return 1 on success, and return 0 on failure.

**Remark:** Set callback function, and replace the OSD information by the user. This function should be called before calling PlayM4_Play, and it will be invalid when calling PlayM4_Stop. The function should be set again for the next call of PlayM4_Play.

Speed is not controlled during decoding a part of the stream. The decoder will decode the other part of stream as long as the stream returned form callback function by the user. The user should know video display and audio play very well, or it is not recommended to use this function. The difference between this callback function and PlayM4_SetDecCallBack is that this callback functionis added user transfer parameters.

The year, month, day, hour, minute, second and millisecond information obtained from callback function, are the global time parsed from the current data stream. The global time is inputted during DSP stream encoding.

**Remark:** Image will not be displayed if this API is called. So it is not recommended to call this API in the current version.

### 5.6.6 Set File End Callback Function

## PlayM4_SetFileEndCallback

*Int PlayM4_SetFileEndCallback (int nPort, void (CALLBACK *FileEndCallback) (int nPort, void *pUser), void *pUser);*

**Description:**

Set callback for decoding file end. This API should be called before PlayM4_OpenSteam () or PlayM4_OpenFile ()

**Parameters:**

**nPort**

[in] Playing port number

**FileEndCallback**

[in] Callback function described below

**pUser**

[in] Parameter of callback function

***Callback function Description:***

*void (CALLBACK*FileEndCallback) (int nPort, void *pUser)*

***Parameters:***

***nPort:***

*Port number*

***pUser:***

*User defined parameters, as the last parameter pUser of PlayM4_SetFileEndCallback ().*

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- For the callback functions, as VB does not support multi-thread, thus problems may occur when calling API defined in VB under VC environment. Please refer to: **Microsoft Knowledge Base Article - Q198607 "PRB: Access Violation in VB Run-Time Using AddressOf "** for detail information.
- PlayM4_SetFileEndMsg and PlayM4_SetFileEndCallback shall not be called at the same time.

## 5.6.7 Set Decoding Key PlayM4_SetSecretKey

*int PlayM4_SetSecretKey (int nPort, int lKeyType, char *pSecretKey, int lKeyLen);*

**Description:**

If a secret key has been set for encryption purpose during the encoding process, then this API shall be called before PlayM4_OpenSteam or PlayM4_OpenFile for decryption.

**Parameters:**

**nPort**

[in] Playing port number

**lKeyType**

[in] secret key type

**pSecretKey**

[in] secret key string

**lKeyLen**

[in] key length (unit: bit)

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.6.8 Set Error Data Skipping Function

### PlayM4_SkipErrorData

int PlayM4_SkipErrorData(int nPort, int bSkip);

**Parameters:**

**nPort**

[in] Playing port number

**nStream**

[in]1. video stream; 2.audio stream; 3.video & audio stream

**Return Values:**

1- API calling succeeds;

0- API calling fails.

**Remarks:**

Set as TRUE if there are errors in the stream.

### 5.6.9 Set Decoding Tolerance Level PlayM4_SetDecodeERC

**API Definition:**

```
int PlayM4_SetDecodeERC
(
   int    nPort,
   int    nLevel
)
```

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No., which is from 0 to 31. |
| *nLevel* | Tolerance level: 1, 2, 3, 4. Higher level indicates higher tolerance capability, but lower decoding performance. If the level is set to 0, it indicates that the tolerance function is disabled. |

**Return Values:** Return *0* for failure.

**Remarks:**

- Setting tolerance level is not supported by hardware decoding, but if it is configured, no error will be returned.
- By default, the tolerance function is enabled with 1 level when decoding Intra flash stream. If the level is set to 0, it will be forced to set to 1. If there is no *intra* marked, error will be returned as there is no I frame when switching to level 0.
- If the decoding tolerance is disabled, you cannot get the stream starts from P frame, otherwise, snow image may appear.

## 5.6.10 Set Number of Decoding Threads

### PlayM4_SetDecodeThreadNumber

**API Definition:** int PlayM4_SetDecodeThreadNumber(
int *nPort,*
int *nThreadNumber*
);

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number, which is between 0 and 31. |
| *nThreadNumber* | [IN] Number of threads |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- The number of threads can be 1 to 8.
- Only the stream with H.264 or H.265 format supports setting number of decoding threads.
- For hardware decoding, setting decoding thread is not supported.

# 5.7 Display Operation

## 5.7.1 Set/Add Display Region PlayM4_SetDisplayRegion

**API Definition:**

```
int    PlayM4_SetDisplayRegion (
       int                nPort,
       unsigned int       nRegionNum,
       RECT               *pSrcRect,
       PLAYM4_HWND        hDestWnd,
       int                bEnable
);
```

**Description:**

Set or add display regions, also applies to digital zoom.

**Parameters:**

**nPort**

[in] Playing port number

**nRegionNum**

[in] Display region number from 0 to (MAX_DISPLAY_WND-1). If nRegionNum is set as 0, it stands for setting of the main display window (window set in PlayM4_Play) and hDestWnd and bEnable settings will not be handled in this mode.

**pSrcRect**

[in] Set the region to be displayed (this region shall be inside the original image), i.e. if the resolution of original image is 352*288, the range of pSrcRect shall not exceed (0, 0, 352, 288).

If pSrcRect is set as NULL, the whole image will be displayed.

**hDestWnd**

[in] Set the display window. If the window for this region has already been set or opend, then this parameter will be neglected.

**bEnable**

[in] Open /set or close the display region.

**Return Values:**

Returns 1 for success, and returns 0 for failure.

**Remark:**

- This API can be called only when the play is started, you can set or add the display area to realize the digital zoom.
- When the nRegionNum is 0, the settings of hDestWnd and bEnable will be ignored and the main window is under process. The value of digital zoom area (pSrcRect) must be larger than 0, and the value of right and bottom must be larger than 16. Otherwise, only display the original image.
- When in hardware decoding mode, the digital zoom function can only be realized in the original window.
- When realizing the digital zoom function, if returns failure, you can still call siwtchtohard to

switch to the hardware decoding.

### 5.7.2 Set Display window PlayM4_SetVideoWindow

**API Definition:**

```
int PlayM4_SetVideoWindow(
    int             nPort,
    unsigned int    nRegionNum,
    PLAYM4_HWND     hWnd
);
```

**Description:**

Set the display window.

**Parameters:**

**nPort**

[in] Playing port number

**nRegionNum**

[in] Display region number from 0 to (MAX_DISPLAY_WND-1).

**hWnd**

[in] The handle of the display window.

**Return Values:**

Returns 1 for success, and returns 0 for failure.

**Remark:**

- If you want to switch the window, you should call this API to set the original window as null, and then call related API to set a new display window.
- The digital zoom will be disabled if this API is called.
- Destroying the original window is supported in hardware decoding mode, and after destroying, the image will keep at the current frame.

### 5.7.3 Set Synchronous Playback Group PlayM4_SetSyncGroup

**API Definition:**

```
int   PlayM4_SetSycGroup(
    int     nPort,
    int     dwGroupIndex
);
```

**Parameters:**

**nPort**

[in] Playing library port

dwGroupIndex

[in] Synchronous playback group No., the value should be between 0 and 3.

**Return Values:**

Returns 1 for success, and returns 0 for failure.

**Remark:**

- You can call this API repeatedly to add the specific nPort to the synchronous playback group.
- Call setSycGroup before calling API to start playing.
- One nGroupIndex indicates one group, up to 16 channels can be added to each group. Otherwise, FALSE will be returned, and you must release the port (call freePort) to remove the channel from the group.
- Global time is needed when in synchronous playback, so only Hikvision devices can be supported.
- When in hardware decoding mode, synchronous playback of multiple channels is also supported. To display one frame, multiple frames should be sent to the decoder, so at the beginning, there is no image.

### 5.7.4 Set Video Rendering Engine

#### PlayM4_SetDisplayEngine

**API Definition:**

```
int PlayM4_SetDisplayEngine(
    int             nPort,
    unsigned int    nDisplayEngine
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number, which is between 0 and 31. |
| *nDisplayEngine* | [IN] Rendering engine parameter, its default value is 3 (OpenGLes). But for setting metal engine, its value should be set to 8. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- This API should be called after getting port No. and before opening file or starting transmitting stream.
- You cannot switch the rendering engine during video playing.
- When setting metal engine, default.metallib should be included, otherwise the engine cannot work.

### 5.7.5 Enable Feature Tracking Effect (for EZVIZ Client)

#### PlayM4_EnableSuperEyeEffect

**API Definition:**

```
int PlayM4_EnableSuperEyeEffect (
    int     nPort,
    int     nRegionNum
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *nRegionNum* | [IN] Display area No., which is between 0 and 5. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- When calling this API, make sure you have opened files, file stream, or real-time stream in which private data of the type 0x1040 is contained.
- When feature tracking effect is enabled, private data of the type 0x1040 will have the priority over those of other types to be matched for each frame.
- The feature tracking effect is the same as that of digital zoom, which is performed according to parameters in private data of each frame.
- Mismatch between private date frame and video frame may occur during the fast forward playback.

## 5.7.6 Disable Feature Tracking Effect (for EZVIZ Client)

### PlayM4_DisableSuperEyeEffect

**API Definition:**

```
int PlayM4_DisabeSuperEyeEffect (
    int     nPort,
    int     nRegionNum
    int     nKeepEffect
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *nRegionNum* | [IN] Display area No., which is between 0 and 5. |
| *nKeepEffect* | [IN] Whether to keep the feature tracking effect. 1-yes; 0-no, the image will turn back to the default effect. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** When calling this API, make sure you have opened files, file stream, or real-time stream in which private data of the type 0x1040 is contained. And the feature tracking effect is enabled.

## 5.7.7 Get Parameters of the Current Display Area

### PlayM4_GetCurrentRegionRect

**API Definition:**

```
int PlayM4_GetCurrentRegionRect (
    int         nPort,
    int         nRegionNum
    HKRECT*     stRect
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *nRegionNum* | [IN] Display area No., which is between 0 and 5. |
| *stRect* | [OUT] Parameters of the current display area. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** When calling this API, make sure you have opened files, file stream, or real-time stream.

### 5.7.8 Set Image Post-processing Parameters

**PlayM4_SetImagePostProcessParameter**

**API Definition:** PlayM4_SetImagePostProcessParameter(
int *nPort*,
int *nType*,
float *fValue*
)

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *nType* | Post-processing type, which is between 1 and 5. |
| *fValue* | Value of the post-processing type. |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- Post-processing types: 1-brightness (value range: [-1.0,1.0]), 2-chromaticity (value range: [0.0,1.0]), 3-saturation (value range: [-1.0,1.0]), 4-contrast (value range: [-1.0,1.0]), 5-sharpness (value range: [0.0,1.0]).
- The post-processing parameters will take effect only for the main image (the first image to be rendered).

## 5.8 Buffer Operation

### 5.8.1 Get Remaining Data Size of Source Buffer

**PlayM4_GetSourceBufferRemain**

*unsigned int PlayM4_GetSourceBufferRemain (int nPort);*

**Description:**

Get the remaining space information of the source buffer.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

Remain space of the source buffer (unit: Byte).

### 5.8.2 Set Maximum Buffer Size PlayM4_SetDisplayBuf

*int PlayM4_SetDisplayBuf (int nPort, unsigned int nNum);*

**Description:**

Set the buffer size for playback (buffer of decoded images). This buffer is directly related with the fluency and real-time performance of playback. Larger buffer size usually means higher

fluency yet inter time delay, and thus it is suggested to set larger buffer size for OpenFile mode (if the system memory is large enough). The default buffer size would be 10 (frames) for Open File mode, which is, about 0.6 sec under cases of 25fps; and 10 (frames) for Open Stream mode.

**Parameters:**

**nPort**

[in] Playing port number

**nNum**

[in] The number of frames to be buffered, range: from MIN_DIS_FRAMES to MAX_DIS_FRAMES

**MIN_DIS_FRAMES**

The minimum number of image frames to be buffered.

**MAX_DIS_FRAMES**

The maximum number of image frames to be buffered.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remark:***

This function shall be called between PlayM4_OpenStream and PlayM4_Play.

### 5.8.3 Get Maximum Buffer Size PlayM4_GetDisplayBuf

**Function:** unsigned int PlayM4_GetDisplayBuf(int nPort)

**Parameters:** int nPort Playing channel No.

**Return Values:** Maximum frame number of the playing buffer

***Remark:*** Return 0 when buffer is not created. Return buffer node number only when the buffer is created.

### 5.8.4 Clear All Buffers PlayM4_ResetSourceBuffer

*int PlayM4_ResetSourceBuffer (int nPort);*

**Description:**

Clear the data in all the buffers.

**Parameters:**

**nPort**

[in] Playing port number

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.8.5 Clear Specified Buffer PlayM4_ResetBuffer

*int PlayM4_ResetBuffer (int nPort, unsigned int nBufType);*

**Description:**

Clear the data in a specified buffer.

**Parameters:**

**nPort**

[in] Playing port number

**nBufType**

[in]

**BUF_VIDEO_SRC:** Video source buffer, valid for stream mode.

**BUF_AUDIO_SRC:** Audio source buffer, valid for video/audio separate stream mode.

**BUF_VIDEO_RENDER:** Video decode buffer.

**BUF_AUDIO_RENDER:** Audio decode buffer.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.8.6 Get specified buffer size PlayM4_GetBufferValue

**API Definition:** unsigned int PlayM4_GetBufferValue(int nPort,unsigned int nBufType)

**Parameters:** [in] nPort

Playing channel No.

[in] nBufType

| Macro Definition | |
|---|---|
| BUF_VIDEO_SRC | Video source buffer, valid for stream mode. Unit: Byte. |
| BUF_AUDIO_SRC | Audio source buffer, valid for video/audio separate stream mode. Unit: Byte. For V7.2.x and later versions, video source and audio source use the same buffer, which similar to BUF_VIDEO_SRC. |
| BUF_VIDEO_RENDER | remained data for video decode buffer (unit: Frame) |
| BUF_AUDIO_DECODED | remained data for audio decode buffer (unit: Frame) |

**Return:** Return 1 on success, and return 0 on failure.

**Remark:** Get the size of playing buffer (unit: frame or byte). BUF_VIDEO_RENDER can be used for estimating network delay time.

***Remark:*** The node number of audio buffer is 60, while the node number of video buffer is 15.

# 5.9 File Indexing

## 5.9.1 Set File Index Callback Function

### PlayM4_SetFileRefCallBack

*int PlayM4_SetFileRefCallBack (int nPort, void ( _stdcall *pFileRefDone) (unsigned int nPort, unsigned int nUser), unsigned int nUser);*

**Description:**

Register a callback function to notify the user when the key frame index for the file is created. If the callback is not triggered, it indicates errors in the file.

The file index is used for fast locating in the file. The indexing speed is about 40MB per second. All the Index-related APIs shall be called after indexing, and the other APIs will not be affected.

**Parameters:**

**nPort**

[in] Playing port number

**SetFileRefCallBack**

[in] pointer of callback function

**nUser**

[in] user data

***Description of the Callback Function:***

***void FileRefDone (unsigned intnPort, unsigned intnUser)***

***Parameters:***

***nPort***

*Playing port number*

***nUser***

*User data*

**Return Values:**

1- API calling succeeds;

0- API calling fails.

# 5.10 Capture

## 5.10.1 Set Display Callback Function

### PlayM4_SetDisplayCallBack

*int PlayM4_SetDisplayCallBack (int nPort, void (CALLBACK* DisplayCBFun) (int nPort, char * pBuf, int nSize, int nWidth, int nHeight, int nStamp, int nType, int nReserved));*

**Description:**

Register a callback function for image capturing.

**Parameters:**

**nPort**

[in] Playing port number

**DisplayCBFun**

[in] Snapshot callback function

***Description of the Callback Function:***

***nPort***

*Playing port number*

***pBuf***

*Pointer of the snapshot buffer*

***nSize***

*Size of the snapshot buffer*

***nWidth***

*Width of the image (unit: pixels)*

***nHeight***

*Height of the image (unit: pixels)*

***nStamp***

*Time stamp (unit: ms)*

***nType***

*Received data type: T_YV12, T_RGB32, T_UYVY, please refer to the macro definition of PlayM4_SetDecCallback() for detail information.*

***nReserved***

*Reserved;*

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- The callback shall return ASAP, as the callback is triggered in clock thread and any time-consuming operation will affect the clock pulse and cause display problems. Users can stop the callback by setting DisplayCDFun as NULL.
- This API can be called at any time, and it will remain valid until application ends.

## 5.10.2 Set Display Callback Function (Extended)

### PlayM4_SetDisplayCallBackEx

Int PlayM4_SetDisplayCallBackEx(int nPort,void (CALLBACK* DisplayCBFun)(DISPLAY_INFO *pstDisplayInfo), int nUser);

**Description:**

Register a callback function for image capturing, this API is an extended version compared to PlayM4_SetDisplayCallback() which contains user data.

**Parameters:**

**nPort**

[in] Playing port number

**DisplayCBFun**

[in] Snapshot callback function

**nUser**

[in] User data

***Description of the Callback Function:***

**nPort**

[in] Playing port number

**pstDisplayInfo**

Pointer of DISPLAY_INFO structure

***Structure Definition:***

```
typedef struct
{
    long    nPort;          //Playing port number
    char * pBuf;            //Pointer of the snapshot buffer
    long    nBufLen;        //Size of the snapshot buffer
    long    nWidth; // Width of the image (unit: pixels)
    long    nHeight;// Height of the image (unit: pixels)
    long    nStamp; // Time stamp (unit: ms)
    long    nType;          //Received data type: T_YV12, T_RGB32, T_UYVY, please refer to the macro definition of PlayM4_SetDecCallback() for detail information.
    long    nUser;          //User Data
}DISPLAY_INFO
```

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- The callback shall return ASAP, as the callback is triggered in clock thread and any time-consuming operation will affect the clock pulse and cause display problems. Users can stop the callback by setting DisplayCDFun as NULL.
- This API can be called at any time, and it will remain valid until application ends.

## 5.10.3 Set Display Callback Function with Global Time PlayM4_RegisterDisplayCallBackEx

**API Definition:**

```
int PlayM4_RegisterDisplayCallBackEx (
    int      nPort,
    void (CALLBACK* DisplayCBFun) (
      DISPLAY_INFO *pstDisplayInfo,
      PLAYM4_SYSTEM_TIME *pstSystemTime,
      int bDettach),
      void* nUser
)
```

**Parameter:**

*nPort*

[in] Playing channel No.

*displayCBEx*

[in] Capture callback function, it can be NULL.

*nUser*

[in] User pointer.

***Callback Function Parameters:***

| | | |
|---|---|---|
| DISPLAY_INFO* | pstDisplayInfo | Playing channel No. |
| PLAYM4_SYSTEM_TIME | *pstSystemTime | Structure pointer for displaying current information. |
| int | bDettach | System time |

**Return Value:** Return *1* on success, and return *0* on failure.

**Remarks:**

- Set capture callback function. Set the callback pointer DisplayCBFun as NULL to stop the callback. The configured callback function will keep valid until PlayM4_Stop is called. This function can be called at any time.
- If the callback is triggered in the clock thread, operations with long time consumption cannot be executed. Otherwise, the clock pulses will be disrupted and the display will be affected.
- The current obtained image data type is YV12 format.
- Call API with nPort parameter in this callback function may cause locked.
- The year, month, day, hour, minute, second and millisecond information obtained from callback function, are the global time parsed from the current data stream. The global time is inputted during DSP stream encoding.

## 5.10.4 Capture BMP Picture PlayM4_GetBMP

**API Definition**

```
int PlayM4_GetBMP (
    int              nPort,
    unsigned char    *pBitmap,
```

```
    unsigned int        nBufSize,
    unsigned short      *pBmpSize
);
```

**Parameters**

**nPort**

[in] Playback channel No.

**pBitmap**

[in] Buffer allocated by users for saving BMP picture. The storage space (sizeof(BITMAPFILEHEADER)+sizeof(BITMAPINFOHEADER)+w*h*4, w and h is the width and height of the picture) should be larger than or equal to the BMP picture size.

**nBufSize**

[in] Applied buffer size

**pBmpSize**

[out] Captured BMP picture size

**Return Values**

Returns 1 for success, and returns 0 for failure.

**Remark**

For video with 2CIF resolution, the resolution of captured picture is 4CIF.

### 5.10.5 Capture BMP Picture with Private Data PlayM4_GetBMPEx

**API Definition**

```
int PlayM4_GetBMPEx(
  int             nPort,
  unsigned char   *pBmp,
  int             nBufSize,
  int             *pBmpSize
)
```

**Parameters**

| | |
|---|---|
| nPort | Playback channel No. |
| pBitmap | Buffer allocated by users for saving BMP picture. The storage space (sizeof(BITMAPFILEHEADER)+sizeof(BITMAPINFOHEADER)+w*h*4, w and h is the width and height of the picture) should be larger than or equal to the BMP picture size. |
| nBufSize | Applied buffer size. |
| pBmpSize | Captured BMP picture size. |

**Return Values** Returns 1 for success, and returns 0 for failure.

**Remark** For video with 2CIF resolution, the resolution of captured picture is 4CIF.

### 5.10.6 Capture JPEG Picture PlayM4_GetJPEG

*int PlayM4_GetJPEG (int nPort, unsigned char* pJpeg, unsigned int nBufSize, unsigned short *pJpegSize);*

**Description:**

Get a snapshot in JPEG format

**Parameters:**

**nPort**

[in] Playing port number

**pJpeg**

[in] [in] Address assigned by users for storing JPEG snapshot, the file size shall be no less then the JPEG file size, suggested value: w * h * 3⁄2, in which w and h stand for the width and height of the image.

**nBufSize**

[in] assigned buffer size

**pBmpSize**

[out]Actual Jpeg image size captured.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remark:***

If the image width or height is not a multiple of 16, the snapshot image resolution will be automatically cropped to multiple of 16, i.e., original image of 176*120 will be cropped to 176*112. The original image resolution of 2CIF file snapshot after the resolution of 4CIF.

### 5.10.7 Transform YUV Data to JPEG Picture

## PlayM4_ConvertToJpegFile

**API Definition:** int PlayM4_ConvertToJpegFile(char* pBuf,int nSize,int nWidth,int nHeight,int nType,char* sFileName);

**Parameters:**

| | | |
|---|---|---|
| char* | pBuf | Image buffer in capture callback function (display the initial address of decoded YUV data obtained from callback function) |
| int | nSize | Image size in the capture callback function |
| int | nWidth | Image width in the capture callback function |
| int | nHeight | Image height in the capture callback function |
| int | nType | Image type in the capture callback function (the current obtained type is YV12) |
| char* | sFileName | File name to save, setting JPG as filename extension is suggested. |

**Return Values:** Return 1 on success, and return 0 on failure.

***Remarks:*** The resolution of picture captured in the 2CIF video is 4 CIF.

The obtained JPEG picture size will be extended to the integral multiples of 16. For example, when playing video with the resolution of 1920 * 1080, the captured picture is 1920 * 1088 (1080 is not the integral multiples of 16). Call this function in the callback function to save the decoded data to JPEG file. The content of pbuf pointer cannot be edited in the function.

## 5.10.8 Set Resolution to Capture BMP Picture

### PlayM4_GetBMPFixPixelEx

**API Definition:**

```
int PlayM4_GetBMPFixPixelEx(
    int             nPort,
    unsigned char   *pBitmap,
    unsigned int    nBufSize,
    unsigned int    *pBmpSize,
    int             nShotWidth,
    int             nShotHeight
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number, which is between 0 and 31. |
| *pBitmap* | [IN] Data input buffer |
| *nBufSize* | [IN] Size of data input buffer |
| *pBmpSize* | [IN] Size of BMP picture |
| *nShotWidth* | [IN] Resolution width |
| *nShotHeight* | [IN] Resolution height |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- The minimum resolution is 16*16 and the maximum resolution is 4096*4096.
- If the input buffer is not configured, you can call this API to get the buffer size, and the size must be larger than or equal to the size of the picture to be captured.

## 5.10.9 Set Resolution to Capture Picture in Fisheye Mode

### PlayM4_FEC_CaptureFixPixel

**API Definition:**

```
int PlayM4_FEC_CaptureFixPixel(
    int             nPort,
    int             nSubPort ,
    unsigned int    nType,
    char            *pFileName,
    int             nBufSize ,
    int             nCapWidth,
    int             nCapHeight
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number, which is between 0 and 31. |
| *nSubPort* | [IN] Fisheye sub port number |
| *nType* | [IN] Captured picture type |
| *pFileName* | [IN] Captured picture buffer |
| *nBufSize* | [IN] Size of data input buffer |
| *nCapWidth* | [IN] Resolution width |

| | | |
|---|---|---|
| | *nCapHeight* | [IN] Resolution height |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- The minimum resolution is 16*16 and the maximum resolution is 4096*4096.
- You can get the buffer size via the API PlayM4_FEC_GetCapPicSizeFixPixel, and the size must be larger than or equal to the size of picture to be captured.

## 5.10.10 Get Size of Picture Captured with Fixed Resolution

## PlayM4_FEC_GetCapPicSizeFixPixel

**API Definition:**

```
int PlayM4_FEC_GetCapPicSizeFixPixel(
    int    nPort,
    int    nSubPort,
    int    *pnBufSize，
    int    nCapwidth,
    int    nCapHeight
);
```

| **Parameters:** | | |
|---|---|---|
| | *nPort* | [IN] Playing port number |
| | *nSubPort* | [IN] Fisheye sub-port number |
| | *pnBufSize* | [IN] Buffer size |
| | *nCapWidth* | [IN] Captured picture width |
| | *nCapHeight* | [IN] Captured picture height |
| | *pnBufSize* | [OUT] Buffer size |

**Return Values:** Return *1* for success, and return *0* for failure.

# 5.11 Hardware Decoding

## 5.11.1 Set Hardware Decoding Priority

### PlayM4_SetHDPriority

**API Definition**

```
int PlayM4_SetHDPriority(
    int         nPort
);
```

**Parameters**

**nPort**

[in] Playing port number

**Return Values**

Returns 1 for success, and returns 0 for failure.

**Remark**

- After setting the priority of hardware decoding, the stream supports hardware decoding will be decoded first, and the other stream will be decoded by software decoding.
- This API should be called after calling API to open stream (PlayM4_OpenStream) and before calling API to start playing (PlayM4_Play).
- Hardware decoding supports H264 and Smart264 encoding (the iOS version should be 8.0 or above).
- Hardware decoding supports H265 and Smart265 encoding (the iOS version should be 11.0 or above).
- If hardware decoding is not supported or failed, it will automatically switch to software decoding.
- Temporary stuck may occur during the process of hardware decoding switching to software decoding.
- iOS hardware decoding supports the function of fisheye, image flip, and digital zoom.
- The hardware decoding is not supported by Intra flash stream.

## 5.11.2 Get Decoding Mode PlayM4_GetDecodeEngine

**API Definition:**

```
int PlayM4_GetDecodeEngine(
    int         nPort
);
```

**Parameters:**

nport

[in] Play channel No.

**Return Values:**

Returns 0 for failure, returns 1 for software decoding, and returns 2 for hardware decoding.

**Remark:**

- Call this API after the stream is playing normally, otherwise, the returned decoding type is invalid.
- When the hardware decoding is enabled, only the functions of digital zoom in original window, BMP capture (including fisheye capture), and fisheye dewarping are supported.

### 5.11.3 Set Whether to Reset Hardware Decoding when Exception Occurs PlayM4_SetResetHardDecodeFlag

**API Definition:** PlayM4_SetResetHardDecodeFlag(
int *nPort*,
bool *bResetFlag*
)

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *bResetFlag* | Whether to reset hardware decoding when exception occurs. |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** This API should be called after PlayM4_SetHDPriority and before playing the stream; otherwise, hardware decoding will be directly switched to software decoding when exception occurs.

### 5.11.4 Set Whether to Enable Multi-slice Detection PlayM4_SetCheckMultiSliceFlag

**API Definition:** PlayM4_SetCheckMultiSliceFlag(
int *nPort*,
bool *bCheckFlag*
)

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number. |
| *bCheckFlag* | Whether to enable multi-slice detection. |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:**

- This API should be called after PlayM4_SetHDPriority and before playing the stream.
- This API only takes effect for playing multi-slice stream after hard decoding.

# 5.12 Pre-recording

## 5.12.1 Set Pre-recording Data Callback Function

### PlayM4_SetPreRecordCallBack

*int PlayM4_SetPreRecordCallBack(int nPort, void (CALLBACK* PreRecordCBfun)(int nPort, void* pData, unsigned int nDataLen, void *pUser), void *pUser)*

**Parameters:**

**nPort**

[in] Playing port number

**PreRecordCBfun**

[in] Data callback function

**pUser**

[in] User pointer

***Description of the Callback Function:***

***nPort***

*Playing port number*

***pData***

*Audio/video data*

***nDataLen***

*Data length*

***pUser***

*User pointer*

**Return Values:**

1- API calling succeeds;

0- API calling fails.

**Remarks:**

- This API should be called before calling PlayM4_Play.
- Do not call any API with the parameter nPort in the callback function, otherwise it may be cause a deadlock.
- The data in callback should be handled timely to avoid blocking the callback function, or it will cause video frame loss.

***Remarks:***

- PlayM4_SetPreRecordCallBack of the current version should be called after the first I frame being displayed. Calling this API after playing for a certain time is recommended.
- The value of the window handle in playing should not be NULL, the window handle must be valid.

***Example Code:***

```
// The local file saving the prerecording data
```

```
HANDLE g_hVidPreFile =
CreateFile("Prereord.mp4",GENERIC_WRITE,FILE_SHARE_WRITE,NULL,CREATE_ALWAYS,FILE_A
TTRIBUTE_NORMAL,NULL);

// Prerecording callback function
void CALLBACK fPreRecordCBfun(long nPort, RECORD_DATA_INFO *pRecordDataInfo,void*
pUser)
  {
    DWORD dwSize = 0;
    if (pRecordDataInfo == NULL)
    {
      return;
    }
    if (pRecordDataInfo->nType)
    {
      if (g_hVidPreFile != NULL)
      {
  // Saving prerecording data
      WriteFile(g_hVidPreFile,pRecordDataInfo->pBuf,pRecordDataInfo->nBufLen,&dwSize,NULL);
        }
      }
  }
  BOOL g_bDecFlag = FALSE;
  // Decoding callback function
  void CALLBACK DecCBFun(long nPort,char * pBuf,long nSize,
                         FRAME_INFO * pFrameInfo,
                         long nReserved1,long /*nReserved2*/)
  {

    CPlayerDlg* pDlg = (CPlayerDlg *)nReserved1;
    DWORD dwSize = 0;
    if (pFrameInfo->nType == T_AUDIO16)
    {
      OutputDebugString("Zytest:: get audio data !\n");
      WriteFile(g_hAudFile,pBuf,nSize,&dwSize,NULL);
    }
     else if ( pFrameInfo->nType == T_YV12 )
    {
      // Mark the decoded first frame.
      g_bDecFlag = TRUE;
    }
  }

    long lPort = -1;
```

```
BOOL g_bFlag = FALSE;
HWND g_hWnd =    NULL;
// Real-time stream callback function of the Network SDK(e.g. real-time stream callback of Hik network SDK)
void CALLBACK fRealDataCallBack_V30(LONG lRealHandle, DWORD dwDataType, BYTE *pBuffer, DWORD dwBufSize, void* pUser)
{
    BOOL bFlag =    TRUE;
    CPlayerDlg* pDlg = (CPlayerDlg*)pUser;

    switch (dwDataType)
    {
    case NET_DVR_SYSHEAD:// Current type: file header
        // Get current playing library port
        bFlag = PlayM4_GetPort(&lPort);
        // Set real-time stream openning mode
        bFlag = PlayM4_SetStreamOpenMode(lPort,1);
        //Open real-time stream data
        bFlag = PlayM4_OpenStream(lPort,pBuffer,dwBufSize,4*1024*1024);
        //Set prerecording mark
        bFlag = PlayM4_SetPreRecordFlag(lPort,TRUE);
        //Set decoding callback, it is used to confirm the first frame being successfully decoded.
        bFlag = PlayM4_SetDecCallBackExMend(lPort,DecCBFun,NULL,0,NULL);
        // Start decoding, this display window must be valid handle, or prerecoding callback function canot be set.
        bFlag = PlayM4_Play(lPort,g_hWnd);
        break;
    case NET_DVR_STREAMDATA:
        // send the data to be decoded.
        bFlag = PlayM4_InputData(lPort,pBuffer,dwBufSize);
        //If the prerecording data callback function hasn't been set before and the first frame is decoded, the prerecord data callback function should be set.
        if ((g_bFlag== FALSE)&&(g_bDecFlag == TRUE))
        {
            Sleep(100);
            OutputDebugString("zytest:: set dec callback success !");
            // Sets prerecording data callback function
            g_bFlag = PlayM4_SetPreRecordCallBack(lPort,fPreRecordCBfun,pUser);
            if (g_bFlag == FALSE)
            {
                DWORD dwErr = PlayM4_GetLastError(lPort);
            }
            else
            {
```

```
                    OutputDebugString("zytest:: set pre record callback success !");
                }
            }
            break;
        }
}
```

***Remarks:***

- *Parameter 1: nPort indicates the acquired port number that distributed within the playing library.*
- *Parameter 2: PreRecordCBfun indicates setting prerecord callback function*
- *Parameter 3: NULL indicates that the value of user data is NULL.*

## 5.12.2 Set Pre-recording Data Callback Function

## PlayM4_SetPreRecordCallBackEx

**Function:**

```
int PlayM4_SetPreRecordCallBackEx(
   int             nPort,
   void (CALLBACK* PreRecordCBfun)(
      int                    nPort,
      RECORD_DATA_INFO*      pRecordDataInfo,
      void                   *pUser
      ),
   void            *pUser
);
```

**Parameters:**

| | | |
|---|---|---|
| int | nPort | Playing channel No. |
| void (CALLBACK* PreRecordCBfun) | | Data callback function |
| void* | pUser | User pointer |
| | | |
| PreRecordCBfun Parameters: | | |
| int | nPort | Playing channel No. |
| RECORD_DATA_INFO * | pData | Structure pointer for pre-recording data. |
| void* | pUser | User pointer |

**Return Values:** Return 1 on success, and return 0 on failure.

**Remarks:** The pre-record function is to transform the device stream to PS package and return to user via callback function. This function is to help the user to save file or execute other operation.

***Remarks:*** There is frame information in the callback data.

## 5.12.3 Set Pre-recording Status PlayM4_SetPreRecordFlag

*int PlayM4_SetPreRecordFlag(int nPort, bool bFlag)*

**Parameters:**

**nPort**

[in] Playing port number

**bFlag**

[in] The flag of pre-recording:

bFlag = 1 means to enable the pre-recording function

bFlag = 0 means to close the pre-recording function

**Return Values:**

1- API calling succeeds;

0- API calling fails.

**Remarks:**

Pre-recording function switches device streams transmitted via network into PS format and return them to users via callback function, thus making it convenient for users restoring files or performing other operation.

# 5.13 Fisheye

## 5.13.1 Enable Fisheye Dewarping PlayM4_FEC_Enable

**API Definition:**
```
int PlayM4_FEC_Enable(
    int nPort
);
```
**Parameters:** *nPort* [IN] Playing port number

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**
- The fisheye interface before version 7.3.1 and that after version 7.3.2 can't be used at the same time.
- The fisheye interface of version 7.3.2 and above supports hard decoding and flipping.
- Recommend to use the fisheye interface of version 7.3.2 and above.
- If the options of **CPU Frame Capture** and **Metal API Validation** in the project are not set to "Disabled", fisheye dewarping cannot be enabled.

## 5.13.2 Disable Fisheye Dewarping PlayM4_FEC_Disable

**API Definition:**
```
int PlayM4_FEC_Disable(
    int    nPort
);
```
**Parameters:** *nPort* [IN] Playing port number

**Return Values:** Return *1* for success, and return *0* for failure.

## 5.13.3 Get Fisheye Dewarping Port PlayM4_FEC_GetPort

**API Definition:**
```
int PlayM4_FEC_GetPort(
  int    nPort,
  int*    nSubPort,
  FECPLACETYPE    emPlaceType,
  FECCORRECTTYPE    emCorrectType
)
```
**Parameters:**

| | |
|---|---|
| *nPort* | Playing channel No. |
| *nSubPort* | Sub window |
| *emPlaceType* | Mounting type, see details in FECPLACETYPE |
| *emCorrectType* | Dewarping type, see details in FECCORRECTTYPE |

**Return Values:** Return 1 for success, and return 0 for failure.

***Remarks:***
- Version 7.3.2 and later support 4 channel expansion, the value range of the sub port is ranging from 2 to 5.

- The fisheye interface of Version 7.3.1 and previous and Version 7.3.2 and later can't be used at the same time.
- Version 7.3.2 and later support hardware decoding and rotation. Version 7.3.2 and later is recommended.
- Digital zoom and fisheye expansion cannot be enabled in the same window.

## 5.13.4 Delete Fisheye Dewarping Port PlayM4_FEC_DelPort

int PlayM4_FEC_GetPort(int nPort , int* nSubPort, FECPLACETYPE emPlaceType , FECCORRECTTYPE emCorrectType)

**Description:**

Get the correction sub port of fisheye stream. The sub port value range is [2~5].

**Parameters:**

**nPort**

[in] Playing port number

**nSubPort**

[in] Sub port

**emPlaceType**

[in] Mounting type

**emCorrectType**

[in] Correction type

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.13.5 Set Fisheye Dewarping Parameters PlayM4_FEC_SetParam

**API Definition:**

```
int PlayM4_FEC_SetParam(
int             nPort ,
int             nSubPort ,
FISHEYEPARAM    *pPara
)
```

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No. |
| *nSubPort* | Fisheye port number |
| *pPara* | A pointer of fisheye dewarping parameter structure, see details in FISHEYEPARAM. |

**Return Values:** Return *0* for failure, and return *1* for success.

**Remarks:**

- The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.
- The absolute values of parameters *fZoom* and *fWideScanOffset* in the

structure FESHEYEPARAM must be larger than or equal to 0.00001, otherwise, there is no effect.

- This API is not available for 3D fisheye dewarping.
- This API can be called before calling PlayM4_FEC_SetWnd to set the display window parameters, and the fisheye dewarping will take effect only when the display window is configured.

## 5.13.6 Get Fisheye Dewarping Parameters

## PlayM4_FEC_GetParam

**API Definition:** int PlayM4_FEC_GetParam(
int *nPort*,
int *nSubPort*,
FESHEYE_PARAM *pPara*
)

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No. |
| *nSubPort* | Fisheye port No. |
| *pPara* | Fisheye dewarping parameter structure |

**Return Values:** Return *true* for success, and return *false* for failure.

**Remarks:** The fisheye API of version 7.3.1 or previous and version 7.3.2 or later can't be used at the same time. The API of version 7.3.2 and later is recommended.

## 5.13.7 Set Display Window PlayM4_FEC_SetWnd

**Function:** int PlayM4_FEC_SetWnd(int nPort , int nSubPort , void * hWnd)

**Parameters:**

| | | |
|---|---|---|
| int | nport | Playing channel No. |
| int* | | Sub window for handling |
| nSubPort | | Set the handle of display window |
| void* | hWnd | |

**Return Values:** Return 1 on success, and return 0 on failure.

**Remarks:**

- The fisheye API of Version 7.3.1 or pervious and Version 7.3.2 or later cannot be called at the same time.
- The fisheye module of Version 7.3.2 or later supports hardware decoding and flip.
- Switching at any time is available.

***Example Code:***

```
//Enable fisheye function
PlayM4_FEC_Enable(m_lPort));
PlayM4_FEC_GetPort(m_lPort,&m_nSubPort1,FEC_PLACE_CEILING,FEC_CORRECT_PTZ));

//Update position and zoom value
//Updateed type
stFEPara1.nUpDateType = FEC_UPDATE_PTZPARAM | FEC_UPDATE_PTZZOOM;
```

```
//PTZ central coordinate
stFEPara1.stPTZParam.fPTZPositionX = 0.2;
stFEPara1.stPTZParam.fPTZPositionY = 0.3;
//PTZ range parameters
stFEPara1.fZoom                    = 0.1;

//stCycleParam is used at the circle center and which will affect all the sub port of current
  fisheye.
PlayM4_FEC_SetParam(m_lPort,m_nSubPort1,&stFEPara1);

//Sub port 2. Ceilling mounting, PTZ correction, gethandling sub port of fisheye dewarping
PlayM4_FEC_GetPort(m_lPort,&m_nSubPort2,FEC_PLACE_CEILING,FEC_CORRECT_PTZ);

//Update position
stFEPara2.nUpDateType = FEC_UPDATE_PTZPARAM;
stFEPara2.stPTZParam.fPTZPositionX = 0.7;
stFEPara2.stPTZParam.fPTZPositionY = 0.8;
PlayM4_FEC_SetParam(m_lPort,m_nSubPort2,&stFEPara2);

//Zoom
stFEPara3.nUpDateType = FEC_UPDATE_PTZZOOM;
stFEPara3.fZoom       = 0.9;

PlayM4_FEC_SetParam(m_lPort,m_nSubPort2,&stFEPara3);

// Update position
stFEPara3.nUpDateType = FEC_UPDATE_PTZPARAM;
stFEPara3.stPTZParam.fPTZPositionX = 0.7;
stFEPara3.stPTZParam.fPTZPositionY = 0.8;
stFEPara3.fZoom                    = 0.9;
PlayM4_FEC_SetParam(m_lPort,m_nSubPort2,&stFEPara3))

//Zoom
stFEPara3.nUpDateType = FEC_UPDATE_PTZZOOM;
PlayM4_FEC_SetParam(m_lPort,m_nSubPort2,&stFEPara3);

PlayM4_FEC_SetWnd(m_lPort,m_nSubPort1,hWnd1);
```

***Remarks:***

*Parameter 1： nPort – The obtained playing port No.*

*Parameter 2： 1- Sub Port No.*

*Parameter 3： hWnd- window handle*

## 5.13.8 Get PTZ Port PlayM4_FEC_GetCurrentPTZPort

**API Definition:** int PlayM4_FEC_GetCurrentPTZPort(
int *nPort*,
bool *bPanorama*,
float *fPositionX*,
float *fPositionY*,
unsigned int **pnPort*
);

**Parameters:**

| | |
|---|---|
| *nPort* | [IN] Playing port number |
| *bPanorama* | [IN] Operation type, “false”-operate the original image; “true”-operate panorama image |
| *fPositionX* | [IN] X-coordinate of the position |
| *fPositionY* | [IN] Y-coordinate of the position |
| *pnPort* | [IN] Sub port number |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:** The fisheye dewarping algorithm is edited, which may affect the calling result of this API.

## 5.13.9 Set PTZ Port PlayM4_FEC_SetCurrentPTZPort

int PlayM4_FEC_SetCurrentPTZPort(int nPort, unsigned int nSubPort)

**Description:**

Set the current sub port of fisheye stream.

**Parameters:**

**nPort**

[in] Playing port number

**pnPort**

[in] Sub port number.

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.13.10 Set PTZ View Frame Type on Fisheye Image PlayM4_FEC_ SetPTZOutLineShowMode

**API Definition:** int PlayM4_FEC_SetPTZOutLineShowMode(
int nPort,
FECSHOWMODE nPTZShowMode
)

| | | |
|---|---|---|
| **Parameters:** | *nPort* | Play channel No. |
| | *nPTZShowMode* | PTZ frame type. |
| **Return Values:** | Return *0* for failure, and return *1* for success. | |
| ***Remarks:*** | FEC_PTZ_OUTLINE_RECT (rectangle frame) is not supported by *nPTZShowMode* | |

### 5.13.11 Get Current PTZ Coordinates

## PlayM4_FEC_PlayM4_FEC_PTZ2Window

int PlayM4_FEC_PTZ2Window( int nPort , int nSubPort ,PTZPARAM stPTZRefOrigin , PTZPARAM stPTZRefWindow , PTZPARAM stPTZWindow , float * fXWindow , float * fYWindow)

**Description:**

Get the current PTZ coordinate value of sub port.

**Parameters:**

**nPort**

[in] Playing port number

**nSubPort**

[in] The desired sub port number

**stPTZRefOrigin**

[in] The current PTZ coordinate value

**stPTZRefWindow**

[in] The initial coordinate value of control area touching by finger or mouse

**stPTZWindow**

[in] The current coordinate value of control area touching by finger or mouse

**fXWindow**

[in] The new X coordinate value of PTZ

**fYWindow**

[in] The new Y coordinate value of PTZ

**Return Values:**

1- API calling succeeds;

0- API calling fails.

### 5.13.12 Enable 3D Rotation PlayM4_FEC_3DRotate

**Function:**

```
int PlayM4_FEC_3DRotate(
  int nPort,
  int nSubPort,
  PLAYM4SRTRANSFERPARAM *pstRotateParam
)
```

| | | |
|---|---|---|
| **Parameters:** | int nPort | Playing Channel No. |
| | int nSubPort | Fisheye sub port No. |
| | PLAYM4SRTRANSFERPARAM* pstFishParam | Set 3D fisheye rotation parameters. |
| **Return:** | Return *1* on success, and return *0* on failure. | |

Notice:

- Fisheye API of version 7.3.1 and previous, and of version 7.3.2 and later cannot be called at the same time.
- Fisheye of version 7.3.2 and later supports hardware decoding and rotation.

The values of parameter fAxisX, fAxisY, and fValue vary with the 3D fisheye dewarping modes, see details as the follows:

1) Value range of rotation around X axis:
   - For asteroid-shaped view: from [-π/2, π/2];
   - For dissected-cylindrical-surface-shaped view: [0, π/4];
   - Rotation ranges of horizontal and vertical surface vary with the value of fValue:
     - When fValue is set to 2.0f, rotation range around X axis:
       - Horizontal-curved-surface-shaped view: [-π/18, π/18];
       - Vertical-curved-surface-shaped view: [-π/6, π/6];
     - When the value of fValue is not 2, the range of rotation value of X axis varies with the value of fValue to ensure that the whole cambered surface can be seen.
   - For others: No limit.
2) Value range of rotation around Y axis:
   - Rotation values of horizontal and vertical surface vary with the value of fValue:
     - When fValue is set to 2.0f, rotation range around Y axis:
       - Horizontal-curved-surface-shaped view: [-π/3, π/3];
       - Vertical-curved-surface-shaped view: [-π/9, π/9];
     - When the value of fValue is not 2, the range of rotation value of Y axis varies with the value of fValue to ensure that the whole cambered surface can be seen.
   - For dissected-cylindrical-surface-shaped view: No limit for rotation value range, but the parameter value should be (-1.0, 1.0).
   - For others: No limit.
3) Zoom:
   - For semisphere-shaped view: [-0.8, 900];
   - For cylindrical-surface-shaped view: [0, 900];
   - For asteroid-shaped view: [-0.3, 1.2];
   - For dissected-cylindrical-surface-shaped view: Not support;
   - For others: No limit.
4) When the value is out of range, the rotation value around X axis is limited for dissected-cylindrical-surface-shaped view (e.g., the range is from 0 to 1, setting -1 means limit to 0, setting 2 means limit to 1); for other effects, the error will be returned.
5) When rotating, the order of setting parameters is Y, X, and Value, so the previous rotation effect will not be affected by the error of the next one (e.g., if the rotation value around Y axis is correct and the X parameter is incorrect, the API will return error, but the image will still rotate according to the Y axis settings).

6) When setting fisheye 3D rotation, if the value is set to threshold value, returned values may vary because accuracy differs in different phones (e.g., if the value is set to -900.7 by the upper layer, the value in Samsung s6 will be -900.700012, while on other platforms the value may be -900.700000).

7) Default values of fisheye 3D rotation:

- For semisphere-shaped view:

fAxisX = 3π/2

fAxisY = 0.0f

fValue = 3.0f

- For cylindrical-surface-shaped:

fAxisX = π/4

fAxisY = 0.0f

fValue = 6.0f

- For dissected-cylindrical-surface-shaped view:

fAxisX = 0.0f

fAxisY = 0.0f

fValue = 0.0f

- For asteroid-shaped view:

fAxisX = π/2

fAxisY = 0.0f

fValue = 1.0f

- For vertical/horizontal-curved-surface-shaped view:

fAxisX = 0.0f

fAxisY = 0.0f

fValue = 2.0f

### 5.13.13 Get Captured Picture Size

## PlayM4_FEC_GetCapPicSize

**Function:**

```
int    PlayM4_FEC_GetCapPicSize(
  int nPort, int nSubPort,
  int* pnBufSize
);
```

**Parameters:**

| | | |
|---|---|---|
| int | nport | Playing channel No. |
| int | nSubPort | Fisheye sub port No. |
| [out]int* | pnBufSize | Set 3D fisheye rotation parameters. |

**Return:** Return 1 on success, and return 0 on failure.

### 5.13.14 Capture in Fisheye Dewarping Mode

## PlayM4_FEC_Capture

**API Definition:**

```
int PlayM4_FEC_Capture(
    int             nPort,
    int             nSubPort,
    unsigned int    nType,
    char            *pPicBuf,
    int             nBufSize
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nSubPort* | Fisheye port number |
| *nType* | Captured picture type (only supports 1-BMP) |
| *pPicBuf* | Buffer used to store picture data (allocated by user) |
| *nBufSize* | Picture size. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- When PlayM4_FEC_Capture is called, the picture captured by fisheye capturing function is the same with the actual displaying image.
- Call PlayM4_FEC_GetCapPicSize to get cache size first, then apply the corresponding cache and transmit it for saving captured picture.
- The captured picture is from the decoding layer, so it may be different with the image in the display window.

### 5.13.15 Set Fisheye Dewarping in Wide Angle Image

## PlayM4_SetImageCorrection

int PlayM4_SetImageCorrection(int nPort, bool bFlag)

**Description:**

Set the image correction.

**Parameters:**

**nPort**

[in] Playing port number

bFlag

[in] The flag of correction

true: enable the image correction;

false: disenable the image correction

**Return Values:**

1- API calling succeeds;

0- API calling fails.

## 5.13.16 Set Fisheye Dewarping Mode

### PlayM4_SetFECDisplayEffect

**API Definition:**
```
int    PlayM4_SetFECDisplayEffect(
   int                  nPort,
   int                  nRegionNum,
   VRDISPLAYEFFECT      enDisplayEffect
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nRegionNum* | Display window No. |
| *enDisplayEffect* | Fisheye dewarping mode, see details in the structure VRDISPLAYEFFECT. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- Before calling this API, the video playing should be started.
- After fisheye view dewarping, the image will restore to normal image.
- The fisheye API in version 7.3.1 or previous cannot be used together with the API in version 7.3.2 or later.
- The fisheye API in version 7.3.2 or later supports hardware decoding and flipping. So they are suggested.

## 5.13.17 Set Fisheye Dewarping Parameters

### PlayM4_SetFECDisplayParam

**API Definition:**
```
int PlayM4_SetFECDisplayParam(
   int                  nPort,
   int                  nRegionNum,
   VRFISHPARAM          *pstFishParam
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nRegionNum* | Display window No. |
| *pstFishParam* | Fisheye dewarping parameters, see details in the structure VRFISHPARAM. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- To realize fisheye dewarping, you should use digital zooming to establish corresponding new window.
- The fisheye API of the version of 7.3.1 and previous versions and 7.3.2 and the later can't be used at the same time.
- The version of 7.3.2 and the later supports hardware decoding and reversal. 7.3.2 and the later are recommended.
- The fisheye dewarping algorithm is edited, so the displayed image is different with the former image.

**Example**

```
int PlayM4_SetFECDisplayParam (int nPort, int nRegionNum ,VRFISHPARAM *pstFishParam);
VRFISHPARAM    pPara = new VRFISHPARAM();
pPara.fRXLeft                = 0.0;
pPara.fRXRight               =1.0;
pPara.fRYTop                 = 0.1;
pPara.fRYBottom              = 0.9;
pPara.fAngle                 = 100;
pPara.fZoom                  = 0.0;
pPara.fPTZX                  = 0.5;
pPara.fPTZY                  = 0.5;
int bFlag = setParam(nPort,1, &pPara);
```

## 5.13.18 Get Fisheye Dewarping Parameters

## PlayM4_GetFECDisplayParam

**API Definition:**

```
int PlayM4_GetFECDisplayParam(
    int             nPort,
    int             nRegionNum,
    VRFISHPARAM     *pstFishParam
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nRegionNum* | Display window No. |
| *pstFishParam* | Fisheye dewarping parameters, see details in the structure VRFISHPARAM. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- The fisheye API of the version of 7.3.1 and previous versions and 7.3.2 and the later can't be used at the same time.
- The version of 7.3.2 and the later supports hardware decoding and reversal. 7.3.2 and the later are recommended.

## 5.13.19 Set Animation in Fisheye Dewarping Mode

## PlayM4_FEC_SetAnimation

**API Definition:**

```
int PlayM4_FEC_SetAnimation
(
    int                 nPort,
    int                 nSubPort,
    VRANIMATIONTYPE     emType,
    int                 nCurFrame,
    int                 nTotalFrames
```

)

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No. |
| *nSubPort* | Fisheye port number |
| *emType* | Animation types for fisheye dewarping mode, see details in the structure VRANIMATIONTYPE. Currently, animation is only supported by the fisheye dewarping type of 180° panorama+curved surface |
| *nCurFrame* | Current animation frame |
| *nTotalFrames* | Total number of animation frames |

**Return Values:** Return *0* for failure, and return *1* for success.

***Remarks:*** This API should be called after starting live view and enabling fisheye dewarping.

## 5.13.20 Set Fisheye Display Callback Function

## PlayM4_FEC_SetDisplayCallback

**API Definition:**

```
int PlayM4_FEC_SetDisplayCallback
(
    int         nPort,
    int         nSubPort,
    CALLBACK*   FECDisplayCallback,
    void        *pUser
)
```

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No. |
| *nSubPort* | Fisheye port number |
| *FECDisplayCallback* | Fisheye display callback function, see details below:<br>void(CALLBACK*FECDisplayCallback)(<br>int nPort,<br>int nSubport,<br>void *pUser<br>) |
| *pUser* | User data |

**Return Values:** Return *0* for failure, and return *1* for success.

***Remarks:*** This API should be called after starting live view and enabling fisheye dewarping.

## 5.13.21 Get Optimal Fisheye View Parameters (Wall Mounting) PlayM4_FEC_3DrotateSpecialView

**API Definition:** PlayM4_FEC_3DrotateSpecialView(

| | |
|---|---|
| int | *nPort*, |
| int | *nSubPort*, |
| int | *nSpecialViewType*, |
| PLAYM4SRTRANSFERPARAM | *pstRotateParam* |

)

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No., ranging from 0 to 31 |
| *nSubPort* | Set fisheye port No. |
| *nSpecialViewType* | Optimal fisheye view |
| *srTransParam* | Rotation configuration structure of 3D fisheye, see details in PLAYM4SRTRANSFERPARAM. |

**Return Values:** Return *1* for success, and return *0* for failure.

## 5.13.22 Set Absolute Rotation Parameters of 3D Fisheye Dewarping PlayM4_FEC_3DrotateAbs

**API Definition:** intPlayM4_FEC_3DrotateAbs (

| | |
|---|---|
| int | *nPort*, |
| int | *nSubPort*, |
| PLAYM4SRTRANSFERPARAM | *pstRotateParam* |

)

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No. |
| *nSubPort* | Set fisheye port No. |
| *pstRotateParam* | Rotation configuration structure of 3D fisheye, see details in PLAYM4SRTRANSFERPARAM. |

**Return Values:** Return *1* for success, and return *0* for failure.

## 5.13.23 Get Absolute Rotation Parameters of 3D Fisheye Dewarping PlayM4_FEC_Get3DRotate

**API Definition:** intPlayM4_FEC_Get3DRotate (

| | |
|---|---|
| int | *nPort*, |
| int | *nSubPort*, |
| PLAYM4SRTRANSFERPARAM | **pstRotateParam* |

)

**Parameters:**

| | |
|---|---|
| *nPort* | Play channel No. |

| | |
|---|---|
| *nSubPort* | Set fisheye port No. |
| *pstRotateParam* | Rotation configuration structure of 3D fisheye, see details in PLAYM4SRTRANSFERPARAM. |

**Return Values:** Return *1* for success, and return *0* for failure.

## 5.13.24 Set Fisheye Dewarping Parameters for EZVIZ Camera PlayM4_SetLDCFlag

**API Definition:**

```
int    PlayM4_SetLDCFlag(
   int     nPort,
   int     nFlag
);
```

**Parameters:**

| | |
|---|---|
| *nPort* | Playing port number |
| *nFlag* | Enable or disable fisheye dewarping of EZVIZ camera. |

**Return Values:** Return *1* for success, and return *0* for failure.

**Remarks:**

- This API is customized for EZVIZ cameras, and it is only available for fisheye stream.
- The fisheye dewarping should be enabled when the image has appeared.

# 5.14 Private Data

## 5.14.1 Render Private Data PlayM4_RenderPrivateData

int PlayM4_RenderPrivateData(int nPort, int nIntelType, int bTrue)

**Description:**

Render the private data.

**Parameters:**

**nPort**

[in] Playing port number

**nIntelType**

[in] The internal type, express by bit. Each bit means a type of private data. For details, please refer to PLAYM4_PRIDATA_RENDER

**bTrue**

[in] Whether to display

**Return Values:**

1- API calling succeeds;

0- API calling fails.

***Remarks:***

- This interface is suitable for stream with smart information. POS text overlay and picture overlay is closed and the other three display is open by default
- Hard decoding support displaying private data.

## 5.14.2 Render Private Data PlayM4_RenderPrivateDataEx

int PlayM4_RenderPrivateDataEx(int nPort, int nIntelType, int nSubType, int bTrue)

**Description:**

Render the private data.

**Parameters:**

**nPort**

[in] Playing port number

**nIntelType**

[in] The internal type, express by bit. Each bit means a type of private data. For details, please refer to PLAYM4_PRIDATA_RENDER

**nSubType**

[in] The sub type. When nIntelType is PLAYM4_RENDER_FIRE_DETCET, nSubType can be PLAYM4_FIRE_ALARM; When nIntelType is PLAYM4_RENDER_TEM, nSubType can be PLAYM4_TEM_FLAG.

**bTrue**

[in] Whether to display

**Return Values:**

1- API calling succeeds;

1- API calling fails.

***Remarks:***

1. This interface is suitable for stream with smart information. POS text overlay and picture overlay is closed and the other three display is open by default
2. Hard decoding support displaying private data.

## 5.14.3 Set Private Data Callback Function

### PlayM4_SetAdditionDataCallBack

**Function:** int PlayM4_SetAdditionDataCallBack(int nPort, unsigned int nSyncType, void (CALLBACK* AdditionDataCBFun)(int nPort, AdditionDataInfo* pstAddDataInfo, void* pUser), void* pUser);

**Parameters:**

| | |
|---|---|
| int nport | Playing channel No. |
| unsigned int nSyncType | Synchronization Type |
| void (CALLBACK* AdditionDataCBFun) | Private data callback function |
| void* pUser | User pointer |

***AdditionDataCBFun Callback Function Parameters***

| | |
|---|---|
| Int nPort | Playing channel No. |
| AdditionDataInfo* pstAddDataInfo | Private data pointer |
| void* pUser | User pointer |

**Return Values:** Return 1 on success, and return 0 on failure.

**Remarks:**

***Remark:*** This API is applicable to streams with intelligent information (such as vehicle data). If there is no intelligent information, the callback function will not respond.

## 5.14.4 Set Private Data Callback Function

### PlayM4_RegisterIVSDrawFunCB

**Function:** int PlayM4_RegisterIVSDrawFunCB(int nPort,void (CALLBACK* IVSDrawFun)(int nPort, PLAYM4_HDC hDC, FRAME_INFO* pFrameInfo, SYNCDATA_INFO* pSyncData,void* dwUser),void* dwUser);

**Parameters:**

| | |
|---|---|
| nport | Playing channel No. |
| IVSDrawFun | Private data picture callback function |
| pUser | User pointer |

***IVSDrawFun Callback function parameters***

| | |
|---|---|
| Int nPort | Playing channel No. |

| | | |
|---|---|---|
| | PLAYM4_HDC hDC | Surface device context |
| | FRAME_INFO* pFrameInfo | Private data information structure pointer pointer |
| | SYNCDATA_INFO* pSyncData | Synchronized data structure |
| | void* dwUser | User pointer |

**Return Values:** Return 1 on success, and return 0 on failure.

**Remark:** Enable private data displaying (enabled by default) before calling this API. Users can get private information via this API and then realize private information display.

***Remarks:*** This API is applicable to streams with intelligent information (related to dot, line and frame).

## 5.14.5 Set Font Library Path PlayM4_SetOverlayPriInfoFlag

**Function:** int PlayM4_SetOverlayPriInfoFlag(int nPort, int nIntelType, int bTrue, char* pFontPath)

**Parameters:**

| | | |
|---|---|---|
| int | nport | Playing channel No. |
| int | nIntelType | Inner type. It is indicated by bit. Each bit refers to a type of private data. For details, refer to PLAYM4_DRAW_FEATURE |
| int | bTrue | Display or not. 1 – Display, 0- Not display |
| char* | pFontPath | Font path. |

**Return Values:** Return 1 on success, and return 0 on failure.

***Remarks:*** This API is applicable for the stream with VCA information. The detection frame in the image can also be captured. Only thermal imaging private data can be supported. After enabling hardware decoding, private data display function is not supported.

## 5.14.6 Set the Background Frame Color of the Private Data with POS Information PlayM4_SetPosBGRectColor

**API Definition:** PlayM4_SetPosBGRectColor(

| | |
|---|---|
| int | *nPort*, |
| PLAYM4_POS_BGRECT_COLOR | *stBGRectColor* |

)

| | | |
|---|---|---|
| **Parameters:** | *nPort* | [IN] Playing port number. |
| | *stBGRectColor* | Structure about color information, see details in PLAYM4_POS_BGRECT_COLOR. |

**Return Values:** Return *true* for success, and return *false* for failure.

***Remarks*** This API is applicable to POS information with background frame.

# 5.4 Reverse Playing

## 5.4.1 Play Reversely PlayM4_ReversePlay

**Function:** int PlayM4_ReversePlay(int nPort)

**Parameters:** int nport Playing channel No.

**Return Values:** Return 1 on success, and return 0 on failure.

***Remarks:***

- No audio played during reverse play. When the resolution is larger than or equal to 1080P, only 25 frames can be stored in the GOP. So when the interval between I frames is larger than 25 frames, the image will be unsmooth.
- You should create file index when playing reversely in the file mode (PlayM4_OpenFile). The supported maximum speed of the reverse play speed is slower than that of the play. TS packaging file is not supported.
- When in stream playing mode (PlayM4_OpenStream), reverse play is not supported by real-time data. For history stream data, i.e., recording files, they will be read and transmitted to playing reversely. The decoded data, which is transmitted in reverse order from application layer, should be a GOP. That is, parse the recording file to GOP, and then transmit the decoded data in reverse order according to the GOP timing sequence. For example, the user can read the recording files in local PC and parse them to GOPs (the frame decoded by I frame), the file time sequence is I1PPPPPPP..., I2PPPPPPP..., I3PPPPPPPP...,....., InPPPPPPP..., while the sequence for transmission is InPPPPP..., In-1PPPPP...,....., I3PPPPPPP..., I2PPPPPP..., I1PPPPPP...

# Chapter 6 Structure

## 6.1 FRAME_INFO

**Description:** Structure about video and audio information

```
struct{
   int              nWidth;
   int              nHeight;
   int              nStamp;
   int              nType;
   int              nFrameRate;
   unsigned int     dwFrameNum;
}FRAME_INFO;
```

**Parameters:**

**nWidth**

Image width in pixels or sound track number

**nHeight**

Image width in pixels or sound track number

**nStamp**

Time stamp in milliseconds

**nType**

The data type. For detail, please refer to *Table 6.1*.

**nFrameRate**

Encoding frame rate or audio sampling rate

**dwFrameNum**

The frame number.

**Interfaces:**

PlayM4_SetDecCallBack、PlayM4_SetDecCallBackMend

**Table 6.1 The description of nType**

| Macro Definition of nType | Definition of nFrameType |
|---|---|
| T_AUDIO16 | T Sampling rate: 16 Khz, Mono, 16 bits |
| T_RGB32 | RGB 32 image, 4 bytes per pixel arranged in 'B-G-R-0…' similar as bitmap (starts from the bottom-left of the image) (Reserved) |
| T_UYVY | uyvy image, arranged in "U0-Y0-V0-Y1-U2-Y2-V2-Y3…." (starts from the bottom-left of the image) format (Reserved) |
| T_YV12 | yv12 image arranged in "Y0-Y1-……""V0-V1….""U0-U1-….." format |

## 6.2 DISPLAY_INFO

**Description:** Structure about display information

```
struct{
   int      nPort;
   char*    pBuf;
   int      nBufLen;
   int      nWidth;
   int      nHeight;
   int      nStamp;
   int      nType;
   void*    nUser;
}DISPLAY_INFO;
```

**Parameters:**

**nPort**

Playing port number

**pBuf**

The pointer of returned image data

**nBufLen**

The size of returned image data

**nWidth**

The width of video

**nHeight**

The height of video

**nStamp**

Time stamp in milliseconds

**nType**

The data type. For detail, please refer to *Table 6.1*.

**nUser**

The user data

**Interfaces:**

PlayM4_SetDisplayCallBackEx

## 6.3 PLAYM4_SYSTEM_TIME

**Description:** Structure about system time.

```
struct{
   unsigned int     dwYear;
   unsigned int     dwMon;
   unsigned int     dwDay;
   unsigned int     dwHour;
   unsigned int     dwMin;
   unsigned int     dwSec;
```

```
    unsigned int      dwMs;
}PLAYM4_SYSTEM_TIME;
```

**Parameters:**

**dwYear**

Year

**dwMon**

Month

**dwDay**

Day

**dwHour**

Hour

**dwMin**

Minute

**dwSec**

Second

**dwMs**

Millisecond

**Interfaces:**

PlayM4_GetSystemTime

## 6.4 HKRECT

Description: Structure about region parameters

```
typedef struct tagHKRect
{
    unsigned long nLef;
    unsigned long nTop;
    unsigned long nRight;
    unsigned long nBottom;
}HKRECT;
```

**Parameters:**

**nLef**

Left

**nTop**

Top

**nRight**

Right

**nBottom**

Bottom

**Interfaces:**

PlayM4_SetDisplayRegion

# 6.5 RECORD_DATA_INFO

**Description:** Structure about pre-record information

typedef struct

{

| | | |
|---|---|---|
| long | nType; | //Data type: 1-file header, 4097-I frame, 4099-P frame, 3-audio stream data, 4-private data |
| long | nStamp; | //Time stamp |
| long | nFrameNum; | //Frame No. |
| int | nBufLen; | //Data size |
| char* | pBuf; | //Frame data, unit: frame |
| PLAYM4_SYSTEM_TIME | stSysTime; | //System time |

}RECORD_DATA_INFO;

**Related API:**

PlayM4_SetPreRecordCallBackEx

# 6.6 LAYM4_PRIDATA_RENDER

**Description:** Structure about private data render types

enum _PLAYM4_PRIDATA_RENDER{

| | |
|---|---|
| PLAYM4_RENDER_ANA_INTEL_DATA | = 0x00000001, |
| PLAYM4_RENDER_MD | = 0x00000002, |
| PLAYM4_RENDER_ADD_POS | = 0x00000004, |
| PLAYM4_RENDER_ADD_PIC | = 0x00000008, |
| PLAYM4_RENDER_FIRE_DETCET | = 0x00000010， |
| PLAYM4_RENDER_TEM | = 0x00000020, |

}PLAYM4_PRIDATA_RENDER

**Parameters:**

**PLAYM4_RENDER_ANA_INTEL_DATA**

VCA (Video Content Analysis)

**PLAYM4_RENDER_MD**

Motion detection

**PLAYM4_RENDER_ADD_POS**

POS text overlay

**PLAYM4_RENDER_ADD_PIC**

Picture overlay

**PLAYM4_RENDER_FIRE_DETCET**

Thermal imaging info

**PLAYM4_RENDER_TEM**

Temperature info

**Interfaces:**

PlayM4_RenderPrivateData

## 6.7 AdditionDataInfo

**Description:** Structure about private data information.

**Structure Definition:**

```
typedef struct
{
    long    lDataType;
    long    lDataStrVersion;
    long    lDataTimeStamp;
    long    lDataLength;
    char*   pData
}AdditionDataInfo;
```

**Members**

| | |
|---|---|
| *lDataType* | Private data type |
| *lDataStrVersion* | Structure version for compatibility |
| *lDataTimeStamp* | Private data time stamp |
| *lDataLength* | Private data size |
| *pData* | Private data pointer |

**Related API:** PlayM4_SetAdditionDataCallBack

## 6.8 SYNCDATA_INFO

**Description:** Structure about the synchronization information of private data.

**Structure Definition:**

```
typedef struct SYNCDATA_INFO
{
   unsigned int     dwDataType;
   unsigned int     dwDataLen;
   unsigned int *   pData;
} SYNCDATA_INFO;
```

**Members**

| | |
|---|---|
| *dwDataType* | Additional information type which is synchronizing to stream data, including VCA information, vehicle-mounted information. |
| *dwDataLen* | Additional information size |
| *pData* | Additional information pointer, e.g., IVS_INFO structure |

**Related API:** PlayM4_RegisterIVSDrawFunCB

## 6.9 PLAYM4_FIRE_ALARM

**Description:** Structure about thermal imaging information.

```
typedef enum _PLAYM4_FIRE_ALARM
{
    PLAYM4_FIRE_FRAME_DIS          = 0x00000001,
    PLAYM4_FIRE_MAX_TEMP           = 0x00000002,
    PLAYM4_FIRE_MAX_TEMP_POSITION  = 0x00000004,
```

```
    PLAYM4_FIRE_DISTANCE                = 0x00000008,
}PLAYM4_FIRE_ALARM;
```

**Parameters:**

**PLAYM4_FIRE_FRAME_DIS**

File source frame display

**PLAYM4_FIRE_MAX_TEMP**

The maximum temperature

**PLAYM4_FIRE_MAX_TEMP_POSITION**

The maximum temperature position display

**PLAYM4_FIRE_DISTANCE**

The maximum temperature distance

**Interfaces:**

PlayM4_RenderPrivateDataEx

# 6.10 PLAYM4_TEM_FLAG

**Description:** Structure about temperature measurement information.

```
typedef enum _PLAYM4_TEM_FLAG
{
    PLAYM4_TEM_REGION_BOX           = 0x00000001,
    PLAYM4_TEM_REGION_LINE          = 0x00000002,
    PLAYM4_TEM_REGION_POINT         = 0x00000004,
}PLAYM4_TEM_FLAG;
```

**Parameters:**

**PLAYM4_TEM_REGION_BOX**

Temperature measurement by box

**PLAYM4_TEM_REGION_LINE**

Temperature measurement by line

**PLAYM4_TEM_REGION_POINT**

Temperature measurement by dot

**Interfaces:**

PlayM4_RenderPrivateDataEx

# 6.11 VRDISPLAYEFFECT

**Description:** Structure about fisheye dewarping display effects.

```
typedef enum tagVRDisplayEffect
{
    VR_ET_NULL                      = 0x100,
    VR_ET_FISH_PTZ_CEILING          = 0x101,
    VR_ET_FISH_PTZ_FLOOR            = 0x102,
    VR_ET_FISH_PTZ_WALL             = 0x103,
    VR_ET_FISH_PANORAMA_CEILING360  = 0x104,
```

```
    VR_ET_FISH_PANORAMA_CEILING180    = 0x105,
    VR_ET_FISH_PANORAMA_FLOOR360      = 0x106,
    VR_ET_FISH_PANORAMA_FLOOR180      = 0x107,
    VR_ET_FISH_LATITUDE_WALL          = 0x108,
    VR_ET_REDBLUE_3D                  = 0x109,
}VRDISPLAYEFFECT;
```

**Parameters:**

**VR_ET_NULL**

Not correction

**VR_ET_FISH_PTZ_CEILING**

Ceiling mounting fisheye

**VR_ET_FISH_PTZ_FLOOR**

Floor mounting fisheye

**VR_ET_FISH_PTZ_WALL**

Wall mounting fisheye

**VR_ET_FISH_PANORAMA_CEILING360**

Ceiling mounting fisheye-1P

**VR_ET_FISH_PANORAMA_CEILING180**

Ceiling mounting fisheye-2P

**VR_ET_FISH_PANORAMA_FLOOR360**

Floor mounting fisheye-1P

**VR_ET_FISH_PANORAMA_FLOOR180**

Floor mounting fisheye-2P

**VR_ET_FISH_LATITUDE_WALL**

Wide angle wall mounting

**VR_ET_REDBLUE_3D**

Red-blue 3D effect

**Interfaces:**

PlayM4_SetFECDisplayEffect

# 6.12 VRFISHPARAM

**Description:** Structure about fisheye dewarping display parameters.

```
typedef struct tagVRFishParam
{
    float fRXLeft;
    float fRXRight;
    float fRYTop;
    float fRYBottom;
    float fAngle;
    float fZoom;
    float fPTZX;
    float fPTZY;
```

}VRFISHPARAM;

**Parameters:**

**fRXLeft**

Horizontal coordinate (min)

**fRXRight**

Horizontal coordinate (max)

**fRYTop**

Vertical coordinate (min)

**fRYBottom**

Vertical coordinate (max)

**fAngle**

180° correction angle

**fZoom**

PTZ correction zoom efficient

**fPTZX**

X coordinate of PTZ correction

**fPTZY**

Y coordinate of PTZ correction

**Interfaces:**

PlayM4_SetFECDisplayParam、PlayM4_GetFECDisplayParam

# 6.13 FECPLACETYPE

Description: Structure about the mounting types of fisheye cameras.

typedef enum tagFECPlaceType

{

FEC_NULL = 0x0

FEC_PLACE_WALL = 0x1,

FEC_PLACE_FLOOR = 0x2,

FEC_PLACE_CEILING = 0x3,

}FECPLACETYPE;

**Parameters:**

**FEC_NULL**

No mounting type

**FEC_PLACE_WALL**

Wall mounting

**FEC_PLACE_FLOOR**

Floor mounting

**FEC_PLACE_CEILING**

Ceiling mounting

**Interfaces:**

PlayM4_FEC_GetPort

# 6.14 FECCORRECTTYPE

**Description:** Structure about fisheye dewarping types.

typedef enum tagFECCorrectType

{

FEC_CORRECT_PTZ = 0x100, //PTZ

FEC_CORRECT_180 = 0x200, //Dual-180° panorama

FEC_CORRECT_360 = 0x300, //360° panorama

FEC_CORRECT_LAT = 0x400 //180° panorama

FEC_CORRECT_SEM = 0x500, //360° panorama+hemispherical surface

FEC_CORRECT_CYC = 0x600, //180° panorama+ cylindrical surface

FEC_CORRECT_PLA = 0x700, //360° panorama+planet extension

FEC_CORRECT_CYC_SPL = 0x800, //180° panorama+ extended cylindrical surface

FEC_CORRECT_ARC = 0x900, //180° panorama+curved surface (wall mounting)

FEC_CORRECT_ARC_VERTICAL=0xa00 //180° panorama+vertical curved surface (wall mounting)

}FECCORRECTTYPE;

**Related API:**

PlayM4_FEC_GetPort

| | **Mounting Type** | | | |
|---|---|---|---|---|
| **Dewarping Type** | FEC_NULL | FEC_PLACE_WALL | FEC_PLACE_FLOOR | FEC_PLACE_CEILING |
| FEC_CORRECT_PTZ | *SR_DE_NULL* | *SR_DE_FISH_PTZ_WALL* | *SR_DE_FISH_PTZ_FLOOR* | *SR_DE_FISH_PTZ_CEILING* |
| FEC_CORRECT_ORIGINAL | | *SR_DE_FISH_ORIGINAL* | *SR_DE_FISH_ORIGINAL* | *SR_DE_FISH_ORIGINAL* |
| FEC_CORRECT_180 | | *Not support* | *SR_DE_FISH_PANORAMA_FLOOR_180* | *SR_DE_FISH_PANORAMA_CEILING_180* |
| FEC_CORRECT_360 | | *SR_DE_FISH_PANORAMA_WALL* | *SR_DE_FISH_PANORAMA_FLOOR_360* | *SR_DE_FISH_PANORAMA_CEILING_360* |
| FEC_CORRECT_LAT | | *SR_DE_FISH_PANORAMA_WALL* | *SR_DE_FISH_PANORAMA_WALL* | *SR_DE_FISH_PANORAMA_WALL* |
| FEC_CORRECT_SEM | | *Not support* | *SR_DE_FISH_SEMISPHERE* | *SR_DE_FISH_SEMISPHERE* |
| FEC_CORRECT_CYC | | *Not support* | *SR_DE_FISH_CYLINDER_FLOOR* | *SR_DE_FISH_CYLINDER_CEILING* |
| FEC_CORRECT_PLA | | *Not support* | *SR_DE_FISH_PLANET* | *SR_DE_FISH_PLANET* |
| FEC_CORRECT_CYC_SPL | | *Not support* | *SR_DE_FISH_CYLINDER_SPLIT_FLOOR* | *SR_DE_FISH_CYLINDER_SPLIT_CEILING* |
| FEC_CORRECT_ARC | | *SR_DE_FISH_ARCSPHERE_HORIZONTAL_WALL* | *Not support* | *Not support* |

| | | | | |
|---|---|---|---|---|
| FEC_CORRECT_ARC_VERTICAL | | *SR_DE_FISH_ARCSPHERE_VERTICAL_WALL* | *Not support* | *Not support* |
| FEC_CORRECT_PANOSPHERE | | *Not support* | *Not support* | *Not support* |

# 6.15 SRDISPLAYEFFECT

**Description:** Structure about display effect of fisheye dewarping.

typedef enum tagSRDisplayEffect

{

SR_DE_NULL = 0x0, ///< No dewarping.

SR_DE_FISH_ORIGINAL = 0x1, ///< Fisheye original view.

SR_DE_FISH_PTZ_CEILING = 0x2, ///< Fisheye PTZ view (ceiling mounting).

SR_DE_FISH_PTZ_FLOOR = 0x3, ///< Fisheye PTZ view (ground mounting).

SR_DE_FISH_PTZ_WALL = 0x4, ///< Fisheye PTZ view (wall mounting).

SR_DE_FISH_PANORAMA_CEILING_360 = 0x5, ///< Fisheye 360° panorama view (ceiling mounting).

SR_DE_FISH_PANORAMA_CEILING_180 = 0x6, ///< Fisheye 180° panorama view (ceiling mounting).

SR_DE_FISH_PANORAMA_FLOOR_360 = 0x7, ///< Fisheye 360° panorama view (ground mounting).

SR_DE_FISH_PANORAMA_FLOOR_180 = 0x8, ///< Fisheye 180° panorama view (ground mounting).

SR_DE_FISH_PANORAMA_WALL = 0x9, ///< Fisheye Panorama view (wall mounting).

SR_DE_FISH_SEMISPHERE = 0xA, ///< Fisheye 3D semisphere view.

SR_DE_EAGLEEYE_SEMISPHERE = 0xB, ///<Eagleeye semisphere view.

SR_DE_EAGLEEYE_PLANE = 0xC, ///< Eagleeye flat view.

SR_DE_FISH_CYLINDER_CEILING = 0xD, ///<Cylindrical-surface-shaped view (ceiling mounting).

SR_DE_FISH_CYLINDER_FLOOR = 0xE, ///< Cylindrical-surface-shaped view (ground mounting).

SR_DE_FISH_CYLINDER_SPLIT_CEILING = 0xF, ///<Dissected-cylindrical-surface-shaped view (ceiling mounting)

SR_DE_FISH_CYLINDER_SPLIT_FLOOR = 0x10, ///< Dissected-cylindrical-surface-shaped view (ground mounting)

SR_DE_FISH_PLANET = 0x11, ///<Asteroid-shaped view

SR_DE_FISH_ARCSPHERE_HORIZONTAL_WALL = 0x12, ///< Horizontal-curved-surface-shaped view (wall mounting)

SR_DE_FISH_ARCSPHERE_VERTICAL_WALL = 0x13, ///<Vertical-curved-surface-shaped view (wall mounting)

SR_DE_FISH_ANIMATION_SWITCH_CEILING = 0x14, ///< Animation switch (ceiling mounting)

```
    SR_DE_FISH_ANIMATION_SWITCH_FLOOR      = 0x15,        ///<Animation switch (ground
mounting)
    SR_DE_PANORAMA_SPHERE                  = 0x16,        ///< Panorama sphere view.
    SR_DE_PANORAMA_PLANET                  = 0x17,        ///< Panorama asteroid-shaped view
}SRDISPLAYEFFECT;
```

# 6.16 VRAnimationType

**Description:** Structure about animation types of fisheye dewarping mode (for wall mounting)

**Structure Definition:**

```
typedef enum tagVRAnimationType
{
    VR_ANIMATION_NULL          =0x0,
    VR_ANIMATION_ARCSPHERE     =0x1
}VRANIMATIONTYPE;
```

| **Members** | | |
|---|---|---|
| | VR_ANIMATION_NULL | None. |
| | VR_ANIMATION_ARCSPHERE | Animation in the fisheye dewarping type of 180° panorama+curved surface |

**Related API:** PlayM4_FEC_SetAnimation

# 6.17 FECSHOWMODE

Description: Structure about PTZ view frame type on fisheye image.

```
typedef enum tagFECShowMode
{
    FEC_PTZ_OUTLINE_NULL,
    FEC_PTZ_OUTLINE_RECT,
    FEC_PTZ_OUTLINE_RANGE,
}FECSHOWMODE;
```

**Parameters:**

**FEC_PTZ_OUTLINE_NULL**

Not display

**FEC_PTZ_OUTLINE_RECT**

Rectangle display

**FEC_PTZ_OUTLINE_RANGE**

Real region display

**Related API:**

PlayM4_FEC_SetPTZOutLineShowMode

# 6.18 FISHEYEPARAM

**Description:** Fisheye dewarping parameter structure

**Structure Definition:** typedef struct tagFECParam

```
{
    unsigned int    nUpDateType;
    unsigned int    nPlaceAndCorrect;
    PTZPARAM        stPTZParam;
    CYCLEPARAM      stCycleParam;
    Float           fZoom;
    Float           fWideScanOffset;
    Int             nResver[16];
}FISHEYEPARAM;
```

| | | |
|---|---|---|
| **Members** | *nUpdateType* | Update types |
| | *nPlaceAndCorrect* | Mounting and dewarping types |
| | *fZoom* | Zooming parameters, range: [0.00001f， 1.0f] |
| | *fWideScanOffset* | Offset of 180˚ and 360˚ panorama, range: [0.00001, 360.0] |
| | *stPTZParam* | PTZ dewarping parameters |
| | *stCycleParam* | Circle center parameters of fisheye view. |
| | *nReser* | Reserved. |

**Related API:** PlayM4_FEC_SetParam, PlayM4_FEC_GetParam

**Remarks:** When setting fisheye parameters, the order to take effect is: PTZ and zooming parameters>only PTZ parameters>circle center parameters>offset parameters>only zooming parameters.

E.g., when the PTZ and circle center parameters are configured, if the PTZ parameters are failed to take effect, both the two configurations are failed; if the PTZ parameters are succeeded to take effect, but the circle center parameters are failed to take effect, only the PTZ configurations are succeeded, and error will be returned by the API.

# 6.19 PTZPARAM

**Description:** PTZ dewarping parameter structure

**Structure Definition:**

```
typedef struct tagPTZParam
{
    float    fPTZPositionX;
    float    fPTZPositionY
}PTZPARAM;
```

| | | |
|---|---|---|
| **Members** | *fPTZPositionX* | X-coordinate of PTZ view center |
| | *fPTZPositionY* | X-coordinate of PTZ view center |

**Related API:** PlayM4_FEC_SetParam, PlayM4_FEC_GetParam

# 6.20 CYCLEPARAM

**Description:** Structure about circle center parameters of fisheye view.

**Structure Definition:**

```
typedef struct tagCycleParam
{
```

```
    float    fRadiusLeft;
    float    fRadiusRight;
    float    fRadiusTop;
    float    fRadiusBottom;
}CYCLEPARAM;
```

**Members**

| | |
|---|---|
| *fRadiusLeft* | X-coordinate of circle left |
| *fRradiusRight* | X-coordinate of circle right |
| *fRadiusTop* | Y-coordinate of circle top |
| *fRadiusBottom* | Y-coordinate of circle bottom |

**Related API:** PlayM4_FEC_SetParam, PlayM4_FEC_GetParam

**Remarks:**

- When two fisheye ports are enabled, and the circle center parameters of one of the ports are configured, if you call PlayM4_FEC_DelPort to delete a fisheye port, the image of the other fisheye port and the original image will be changed.
- For fisheye view's circle center, the default values and parameter ranges are as follows:

| Dewarping type | Default value | Parameter range |
|---|---|---|
| Horizontal-curved surface | Left: 0.05<br>Right: 0.95<br>Top: -0.3<br>Bottom: 1.3 | Left: (-0.5, 0.4)<br>Right: (0.6, 1.5)<br>Top: (-0.5, 0]<br>Bottom: [1, 1.5) |
| Vertical-curved surface | Left: -0.16<br>Right: 1.16<br>Top: 0.005<br>Bottom: 0.995 | Left: (-0.5, 0]<br>Right: [1, 1.5)<br>Top: (-0.5, 0.4)<br>Bottom: (0.6, 1.5) |
| Others (non-curved surface) | (0, 1, 0, 1) | Left/ (or)Top: (-0.5, 0.4)<br>Right/ Bottom: (0.6, 1.5)<br>Left+Right/Top+Bottom: (0.8, 1.2) |

Note: By default, the value of circle center is (0,1,0,1) before Metal is transmitted.

## 6.21 PLAYM4SRTRANSFERPARAM

**Description:** Structure about rotation parameters of 3D fisheye

**Structure Definition:** typedef struct tagPLAYM4SRTRANSFERPARAM

{
PLAYM4SRTRANSFERELEMENT *pTransformElement;
unsigned int nTransfromCount
}PLAYM4SRTRANSFERPARAM;

**Members**

*pTransformElement* Rotation coordinates, see details in PLAYM4SRTRANSFERELEMENT

*pTransformElement* Number of rotations

**Related API:** PlayM4_FEC_3DrotateSpecialView

# 6.22 PLAYM4SRTRANSFERELEMENT

**Description:** Structure about rotation unit parameters

**Structure Definition:** typedef struct tagPLAYM4SRTRANSFERELEMENT

{
float *fAxisX*;
float fAxisY*;*
float *fAxisZ;*
float *fValue;*
}

**Members**

*fAxisX* Value of rotating around X-axis

*fAxisY* Value of rotating around Y-axis

*fAxisZ* Value of rotating around Z-axis

*fValue* Zoom in (-) or out (+)

**Related API:** PlayM4_FEC_3DrotateSpecialView

**Remarks:**

- The configuration order is *fAxisY, fAxisX*, and *fValue*, the later configured parameter will not affect the former configured parameters, e.g., if the configured *fAxisY* is correct, but the *fAxisX* is wrong, the image will rotate around the Y-axis, but error will be returned.
- For *fAxisX* , if the configured value is out of the limitation, the value will be automatically set to the limitation, but for other parameter, if the value is out of limitation, error will be returned.

# 6.23 DemuxParam

**Description:** Demux parsing parameter structure.

**Structure Definition:** typedef struct{
int *nDemuxType*;
int *nChunSize*;
}DemuxParam;

**Members**

*nDemuxType*; Parsing type, which must be set to “1”

*nChunSiz* Chunk size.

**Related API:** PlayM4_SetDemuxParam

# 6.24 PLAYM4_POS_BGRECT_COLOR

**Description:** Color information structure.

**Structure Definition:**

```
typedef struct _PLAYM4_POS_BGRECT_COLOR_{
unsigned char nA;
unsigned char nR;
unsigned char nG;
unsigned char nB;
}PLAYM4_POS_BGRECT_COLOR;
```

**Members**

| | |
|---|---|
| *nA* | Alpha color component (transparency), the value is between 0 and 100. |
| *nR* | Red color component, the value is between 0 and 255. |
| *nG* | Green color component, the value is between 0 and 255. |
| *nB* | Blue color component, the value is between 0 and 255. |

**Related API:** PlayM4_SetPosBGRectColor

# Chapter 7 FAQ

## 7.1 General

### 7.1.1 What platforms are covered by the Mobile Platform PlayCtrl Library, and what operating system version is required for each one?

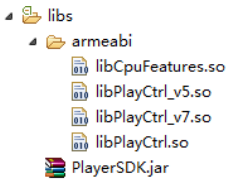
***Answers:***

- For Android PlayCtrl Library, it requires Android 2.3 and above.
- For iOS PlayCtrl Library, it requires iOS 6.0 and above.
- For Windows Phone PlayCtrl Library, it requires Wp8 and above and currently it is not available.
- The performance data will be enhanced when checking performance in iOS12 by using Xcode10.

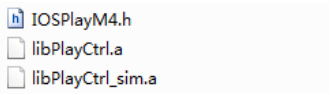
### 7.1.2 What is the file organization of each platform of the Mobile Platform PlayCtrl Library?

***Answers:***

- For Android PlayCtrl Library, it contains one jar package and four library files. You can select one PlayCtrl file according to the CPU type. The default storage directory is app project directory.




- For iOS PlayCtrl Library, it contains one header file and two library files. The library file with sim suffix is the simulator version (optional).




### 7.1.3 How to deal with the problem when live view or playback shows black image?

***Answers:***

***Steps:***

1. Check if the operating system version meets the requirements.
2. Check if the stream is the encryption stream and if the key is correct.
3. Check if the stream is correct by using the Hik PC client and PlayCtrl Library.
4. Check if there is a “No Configs for your device!” log by using the logcat tool.

### 7.1.4 How to deal with the problems for the special phone model?

***Answers:***

- Note down the phone system version.
- Note down the phone model info, including brands, distribution areas, chip information, etc., the more detailed, the better.
- Save the stream and screenshot if the phone shows blurred screen.

### 7.1.5 How to deal with the problem that Android PlayCtrl Library flash exit?

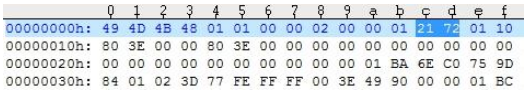
***Answers:***

- Check if the library file is missing, especially libCpuFreatrue.so.
- Check if the phone CPU is the arm architecture, non x86.
- Check the flash exit log by using the logcat tool.

### 7.1.6 How to deal with the problem that there is no sound during playing?

***Answers:***

- Check if the stream contains audio by using Hik PC client and PlayCtrl Library.
- If the audio format is G722, you can check the 40 byte file header after recording. The 13th and 14th byte represents the audio format (c and d in the following picture), together is 0x7221. Some old devices may be labeled as 0x1011.



# 7.2 Live View in Stream Mode

## 7.2.1 Why is the live view stuck?

***Answers:***

***Steps:***

1. Check if the data is fluent, if not, please check the network.
2. Reduce the live view channels properly when multiple channels HD live view is stuck.
3. Check the current resolution. If the buffer is too small, the playing buffer with the input data from PlayM4_InputData may be full. You need to repeatedly input the data that is not successfully input.

   ***Suggestions:***

   - If the resolution is 4CIF, the PlayM4_OpenStream buffer can be set to 1.5~2M;
   - If the resolution is 720P, the buffer can be set to 3~4M；
   - If the resolution is 1080P, the buffer can be set to 6M;

   ***Note:*** 1M is equal 1*1024*1024.
4. Check PlayM4_SetDisplayBuf, you can set 15 nodes.
5. Save the data and play. If it is stuck, please check if some data is missing in network; If it is normal, please check if the input data is right.

## 7.2.2 Why is there no image displayed during live view?

***Answers:***

***Steps:***

1. Check if the correct data is input into the playing buffer and decodes by calling PlayM4_Play.
2. Check if the callback function PlayM4_SetDecCallBack is registered that doesn't show image when decoding.
3. Check if the snapshot is successful. If not, please refer to step 1, or refer to step 4.
4. If it is successful, please snapshot continuously. If the pictures change with the scene and OSD, the data is normal and check if the screen handle is correct.
5. Save the decoding data as the local file and play. If it can be played, please refer to step 6, or refer to step 7.
6. If it can be played normally, please check if the input data of PlayM4_InputData is correct.
7. If it can't be played normally, please check if the data stream is correct. If it is the standard stream, you can use the third party decoder such as Elecard or VLC to verify. If the decoding is successful, please send the test stream to us to analyze. If not, please check if there is network package loss.
8. If the real time stream is in RTP format, please provide network package files in the following format: *the length of 4 byte RTP package + RTP data package* and send them to us.

### 7.2.3 Why does the live view delay?

***Answers:***

1. For the general devices, the delay is 100~200ms and the delay time has a big relationship with the network fluency. If the delay is normal at full frame rate and abnormal at low frame rate, please send the feedback to us. The delay is the minimum after version 6.2.2.2.
2. If the delay appears when multi-channel live view, please check the current CPU usage. If the CPU usage is at limitation (85% and above), please reduce some channels' live view or improve the hardware performance.
3. If the customer is using client software, you can set it good fluency. If the customer use SDK to secondary develop, you can call interface PlayM4_SetDisplayBuf to set the frame number of decoding buffer.
4. Set the interface PlayM4_SetStreamOpenMode to STREAM_REALTIME.
5. Check the network is good and the delay time of sending and receiving network packets is small.

### 7.2.4 Why does the first frame display slower than other frames in the live view?

***Answers:***

- Check if the first frame is I-frame, if not, it needs to wait until I-frame to decode normally. Therefore, it needs long time.
- Poor network environment may cause this problem when real time stream live view.
- When enabling Hard-Decoding Preferred, the stream meets the requirements. But when decoding has errors, it needs to change Soft-Decoding which may also cause this problem.

### 7.2.5 Why can't I play the encrypted stream?

***Answers:***

***Steps:***

1. Check if the stream is the baseline stream. If it is the custom stream, you need to contact the branch company to get the encryption type and the order number.
2. Check if the key is correct and the key setting interface PlayM4_SetSecretKey is calling properly.
3. You can send the stream to us.

### 7.2.6 How to store the snapshot for the follow-up processing?

***Answer:***

After decoding and playing normally (PlayM4_Play), you can call API (PlayM4_GetBMP and PlayM4_GetJPEG) to store the captured picture.

### 7.2.7 Why is it stuck when playing audio, but fluent when playing video?

***Answer:***

This problem may exist in the low frame rate files or it may be caused by the uniform audio. Check if the audio data is uniform.

### 7.2.8 Why is there no sound when playing file?

When playing the files, there is no sound.

***Answers:***

1. Check if there is audio data in stream and the audio has sound.
2. The audio package related type fields don't meet the rule, such as payload error in RTP package, stream type error in PS package, etc.
3. The audio is not encrypted but is encrypted in the package layer. After PlayCtrl Library 6.2 version, it begins to focus on the audio encryption filed. If the previous version can play the audio normally, but the later version doesn't work, this may be the device problem. You need to check with the device.

### 7.2.9 Why does the fast forward occur when playing file?

***Answer:***

Check if the frame rate or the timestamp of the stream is normal. You can also check if there is frame extracting.

### 7.2.10 Failed to open files

***Answers:***

- Check if the file header is correctly saved when saving files. For the device network SDK, the stream type in the real time stream callback function is NET_DVR_SYSHEAD. For the compression card, the current type in the encoding callback function is PktSysHeader.
- Check if the size of the file is less than 4K or greater than 4G.
- Check if there is I-frame. Video decoding relies on I-frame.

### 7.2.11 How to locate the file mode?

***Answer:***

Check if the file index is created successfully. If it is successful, check if the locating parameters such as time or frame number are set correctly.

### 7.2.12 How to get the data decoded from multi-channel data stream?

***Answers:***

***Steps:***

1. One channel data corresponds to one playing port (get from PlayM4_GetPort).
2. Call PlayM4_SetStreamOpenMode to set the correct play mode.
3. Input the file header of this channel data into PlayM4_OpenStream.
4. Call the register decoding callback function PlayM4_SetDecCallBack to get the decoded data.
5. Call PlayM4_Play to start the decoding thread. If you don't need to display the image, you can set screen handle NULL.
6. Call PlayM4_InputData to input the correct data for each channel. The PlayCtrl Library supports multiple concurrent decoding and the current version supports up to 500 channels.

### 7.2.13 How to avoid frame loss when calling decoding callback function in live view?

***Answers:***

1. Real time record files and check if the data stream has some problems.
2. If the data has no problem, please comment all the implements in the decoding callback functions and check if there is frame loss. Also, the operation is more time-consuming when the decoding functions call each other which may cause this problem.
3. Check if PlayM4_InputData has failed during the actual calling. If it has failed, please try to input the data again. If the error frequency is high, please set the size of buffer in PlayM4_OpenStream bigger.

### 7.2.14 Why does the video frame obtained from decoding callback function only display a half height?

***Answer:***

The resolution of original image is 2CIF which means 704*288 for PAL standard and 704*240 for NTSC standard. But the actual display is 704*576 for PAL standard and 704*480 for NTSC standard. In the decoding callback function, it gets the actual resolution of stream which causes this problem.

### 7.2.15 Why does the error occur when calling PlayM4_OpenStream?

Answers:

Steps:

1. Check if the parameters are correct. When calling the interface PlayM4_OpenStream, correct file header needs to be input (The correct file header content and file header length is NET_DVR_SYSHEAD obtained from the real time stream callback function in the network SDK and the PktSysHeader type obtained from the encoding callback function in compression card )
2. When calling failed and the returned error code is PLAYM4_ALLOC_MEMORY_ERROR, it means the memory is not enough. You can check the current PC memory usage.

After the flow data operation, you can call PlayM4_CloseStream interface to close the flow recovery system resource. PlayM4_CloseStream and PlayM4_OpenStream need to appear in pairs.

First Choice for Security Professionals

www.hikvision.com